Contents

1 Introduction **4**
- 1.1 Motivation . . . 4
- 1.2 Problem . . . 4
- 1.3 Gap . . . 5
- 1.4 The compared strategies . . . 6
- 1.5 Research questions . . . 6
- 1.6 Contributions . . . 7
- 1.7 Scope . . . 8
- 1.8 Outline . . . 8

2 Background and Related Work **10**
- 2.1 Multirotor dynamics and baseline control . . . 10
- 2.2 Aerodynamic interaction in multirotor teams . . . 11
- 2.3 Reactive compensation: measuring the residual online . . . 12
- 2.4 Predictive compensation: learning the residual from relative state . . . 13
- 2.5 Injecting the prediction: the controller matters . . . 15
- 2.6 Learning the compensation directly . . . 17
- 2.7 Hybrid reactive and predictive compensation . . . 17
- 2.8 How these methods are evaluated . . . 18
- 2.9 Summary and gap . . . 20

3 System Modelling and Residual-Force Formulation **23**
- 3.1 Notation and conventions . . . 23
- 3.2 Rigid-body model with a residual wrench . . . 24
- 3.3 Measuring the residual . . . 24
  - 3.3.1 What this measurement requires, and how it fails . . . 25
  - 3.3.2 Sign convention in the control law . . . 26
- 3.4 Predicting the residual from relative state . . . 27
  - 3.4.1 Deep-sets parameterisation . . . 27
  - 3.4.2 The superposition assumption . . . 28
  - 3.4.3 Input conditioning . . . 29
  - 3.4.4 Normalisation . . . 30
- 3.5 Assumptions and where they fail . . . 30
- 3.6 Summary . . . 30

4 Control Architectures **32**
- 4.1 The common controller . . . 32
  - 4.1.1 Position loop . . . 32
  - 4.1.2 Attitude loop . . . 33

4.1.3 Parameters . . . 33
4.1.4 Rates . . . 34
4.1.5 Runtime selection . . . 34
4.2 Strategy 0: uncompensated geometric baseline . . . 35
4.3 Strategy 1: INDI . . . 35
4.4 Strategy 2: geometric with a learned residual . . . 37
4.5 Strategy 3: INDI with a learned residual . . . 38
4.6 Strategy 4: feedback linearisation with a flatness-preserving residual . . . 39
4.7 What makes the comparison fair, and what it does not guarantee . . . 39
4.8 Summary . . . 40

5 Experimental Setup and Protocol 41
5.1 Platform . . . 41
5.2 Flight volume . . . 42
5.3 The formation library . . . 42
5.3.1 The null scenarios . . . 42
5.3.2 Regimes, and why the library spans them . . . 43
5.4 Measurement and logging . . . 43
5.4.1 Why the dataset is recorded onboard . . . 43
5.4.2 What is recorded . . . 44
5.4.3 Time alignment across vehicles . . . 44
5.5 Protocol . . . 44
5.5.1 Hardware gate . . . 44
5.5.2 Data collection . . . 45
5.5.3 Comparison campaign . . . 45
5.5.4 Metrics . . . 46
5.5.5 Statistical treatment . . . 47
5.6 The role of simulation . . . 47
5.7 Threats to validity . . . 47
5.8 Summary . . . 48

6 Two-Robot Experimental Results 52
6.1 Experimental campaign overview . . . 52
6.2 Tracking and attitude . . . 53
6.3 Control effort and safety . . . 53
6.4 Predictive strategy (Strategy 2) . . . 54
6.5 Loop attribution . . . 54
6.6 Hybrid strategy (Strategy 3) . . . 54
6.7 Summary — when which strategy . . . 55

7 Multi-Robot and Scaling Results 56
7.1 Three-robot comparison campaign . . . 56
7.2 Null scenarios and the measurement floor . . . 56
7.3 Transfer penalty for the summed predictor . . . 56
7.4 Extension to four and five vehicles . . . 57
7.5 Limitation if three-robot data are incomplete . . . 57
7.6 Chapter summary . . . 57

8 Discussion **58**
8.1 RQ1 — Reactive versus predictive compensation under identical conditions 58
8.2 RQ2 — Regime dependence . . . 58
8.3 RQ3 — Two-robot training on three-robot formations . . . 58
8.4 RQ4 — Cost versus performance . . . 59
8.5 Contributions revisited . . . 59
8.6 Threats to validity, revisited . . . 59
8.7 The modelling assumptions, revisited . . . 60
8.8 The uncompensated moment residual . . . 60
8.9 Frozen gains and the scope of the comparison . . . 60

9 Conclusion **61**
9.1 Summary . . . 61
9.2 Answers to the research questions . . . 61
9.3 Engineering takeaway . . . 62
9.4 Future work . . . 62
9.5 Closing remark . . . 62

Chapter 1

Introduction

1.1 Motivation

When several quadrotors fly close together, the downwash of the upper vehicle (the air its rotors push down) can destabilise the one below. The force is not large in absolute terms: for two Crazyflies hovering 30 cm apart vertically it amounts to about 9 g of thrust, well within their thrust capability [14]. The difficulty lies in its structure, described below. The usual remedy is to stay far enough apart that the interaction can be ignored. For a Crazyflie 2.0 (about 9 cm rotor-to-rotor, vendor data sheet) that conservative planning gap has been taken as about 60 cm; Shi et al. [14] cite that figure from earlier planning work as one such example.

That margin is expensive. It sets the density at which a team can operate, and therefore the size of the space a given number of vehicles requires. Applications that motivate multirotor teams in the first place — inspection in confined spaces, cooperative transport, aerial docking — are precisely those in which the space is not available. In some of them, close approach is the task rather than a hazard to be avoided.

Removing the margin requires a controller that accounts for the interaction rather than avoiding it, and that is difficult for a specific reason. The interaction is not a small perturbation around a known model: it is strongly nonlinear in the relative configuration of the team, growing sharply as vehicles close and decaying quickly with lateral offset, and it is markedly asymmetric, since the vehicle above is affected far less than the one below [8, 4]. It is also hard to derive from first principles in the regime that matters. Simplified models exist for part of the problem: the mean flow from a quadrotor's propellers in hover is well approximated by a turbulent jet beyond about two and a half motor-to-motor distances below the vehicle [1]. Formation flight happens closer than that. In that near field those authors report no closed-form model.

The field's response has been to obtain the interaction from data rather than from theory, and it has worked. Reported results include a reduction in worst-case height error by a factor of two or better, up to four [14], a heterogeneous team of sixteen vehicles flying at a minimum vertical separation of 24 cm [13], and, as one abstract states them, average reductions of 56 % in vertical and 36 % in three-dimensional tracking error for a pair of vehicles [16]. The margin has been substantially reduced.

1.2 Problem

A controller can obtain information about the interaction force in two broad ways, and the distinction organises much of the field.

It can **measure** the residual online, by comparing the acceleration the vehicle actually experienced against the acceleration its nominal model predicts for the actuation it applied. This requires no prior knowledge of the interaction and works equally against disturbances never seen before. But the estimate exists only after the disturbance has begun to act, and in its established form it depends on rotor-speed sensing.

Or it can **predict** the residual before it acts, from the relative states of its neighbours, using a model learned offline, or, where one exists, a physics-based model. This can compensate ahead of the disturbance rather than after it. The model is only valid over the configurations its training data covered. Outside them its accuracy is not merely degraded but unknown: an implementation can bound the size of its output, but not make that output correct.

The two sources are complementary. Measurement is late but not tied to a training set. Prediction can act earlier but only where it was trained. Combining them is the obvious next construction. It has been built with INDI on a single vehicle. There the authors describe the extra gain over the reactive method as small without a payload, and as leaving performance similar to the reactive method when a payload is attached [3]. On a team of three it has been built inside an off-board model predictive controller, combining an online estimate with a learned downwash model, and there the combination performed best [6].

The practical question a system designer faces is which to use, and it has no answer in the literature. Not because the individual results are weak, and not because the field avoids systematic comparison. Recent work benchmarks online-learning estimators on a single vehicle [5]. It is that the results for *these* families cannot be composed: each is reported against its own baseline, on its own airframe, at its own separations, on its own trajectories, under its own metrics.

1.3 Gap

Representative methods exist for reactive compensation, for predictive compensation and for their hybrid, and each is validated on its own. To the best of our knowledge, no study has compared rotor-speed-based reactive compensation, learned per-neighbour predictive compensation and their combination, with the measured and the predicted term added, as switch settings of one onboard controller on one airframe, across a fixed library of interaction regimes, with one definition of the residual for every flight. The closest study compares an online $\mathcal{L}_1$ estimate, a learned downwash model and their combination inside one off-board model predictive controller on three vehicles in two stacked formations [6]. The field's reporting conventions make such a comparison impossible to assemble after the fact. This thesis compares all three, the reactive, the predictive and the hybrid strategy, as switch settings of one controller. A 36 % improvement measured on two vehicles in close proximity [16] and a 31 % improvement reported in the abstract of a tight-formation study [7] are not the same quantity. Neither can be subtracted from the other, and their ordering says nothing about which method would prevail if both were flown under the same conditions.

This is not a criticism of any individual work. Each is internally valid, and none violated a convention of the field. It is a property of the literature taken as a whole: the results were not designed to compose, and establishing a ranking requires an experiment built for that purpose.

Two narrower gaps follow. The INDI-based hybrid has been demonstrated only where the residual is *self-induced*: a single vehicle, whose residual comes from its own unmodelled

dynamics — the authors list blade flapping (the up-and-down motion of the rotor blades in forward flight), drag and ground forces among the sources of such residuals — and, optionally, a slung payload (a payload hanging below the vehicle on a cable) [3]. On a team, measurement and prediction have been combined only inside an off-board model predictive controller [6]; an INDI-based hybrid has not been shown where other vehicles produce the residual and relative state is measurable. That is where the predictive component has the most to contribute, and this thesis evaluates such a hybrid on a team. The summation that predictive architectures use to aggregate per-neighbour contributions is known to be an approximation [14, 4]. The size of the penalty on a deployed model is not reported.

1.4 The compared strategies

Five control strategies are defined. Strategies 0–3 are one controller, a geometric tracking controller to which three independent switches add a measured residual force, a measured residual moment and a learned residual force; they are compared. Strategy 4 is a separate controller. Chapter 4 describes them in detail.

Strategy 0, geometric baseline. The geometric tracking controller with all switches off; it does not use the residual. It is the reference against which the other strategies are compared.

Strategy 1, INDI. Incremental nonlinear dynamic inversion (INDI). It measures the residual force and moment online, from the acceleration the vehicle actually had and the rotor speeds, and corrects for them. This is the reactive strategy.

Strategy 2, geometric + neural network (NN). The geometric controller plus a neural network that predicts the vertical component of the residual from the states of the neighbouring vehicles. This is the predictive strategy.

Strategy 3, INDI + NN. The measured residual of Strategy 1 and the prediction of Strategy 2, added together. This is the hybrid strategy.

Strategy 4, feedback linearisation (FBL) + NN. A feedback-linearising controller with the same kind of prediction. It is not implemented in this thesis; it is described as a reference design, to be added in a later tier of the comparison if it is implemented, and otherwise stated as a limitation.

1.5 Research questions

RQ1 Under identical hardware, trajectories, formations and separations, how do reactive and predictive interaction-force compensation strategies compare to the uncompensated baseline in trajectory-tracking accuracy and residual-force rejection?

RQ2 Does the relative advantage of reactive versus predictive compensation depend on the interaction regime, separation, relative velocity, and whether the disturbance is quasi-static or transient?

RQ3 How large is the transfer penalty when a summed per-neighbour residual model is trained only on two-vehicle data and then used on three-vehicle formations, and is that penalty small enough that the summed architecture remains the right engineering choice? This measures the size of a known approximation. It is not a test of

whether superposition holds: it does not, and that is already established [14, 4]. The question applies to the strategies with a learned prediction (Strategies 2 and 3, and Strategy 4 if implemented). It does not apply to a purely reactive law. RQ3 is answered in the second, three-vehicle phase of the study (Chapter 5).

RQ4 For each compared strategy, what sensing, onboard compute, training data and tuning effort does it require, and how do those costs sit next to the tracking and rejection it delivers?

RQ1 is the primary question and the one the experimental design serves. RQ2 carries the most engineering value, because it converts a ranking into a rule about when to use which. RQ3 asks for the magnitude of the transfer penalty on a deployed model, not whether superposition holds. RQ4 compares cost against performance for every compared strategy.

Each question is answerable by the experiments described in Chapter 5, which has two phases: a two-vehicle campaign for RQ1, RQ2 and RQ4, and then the three-vehicle scenarios, flown with the same controllers, for RQ3 and for the measurement floor. A negative answer to any of them is reported as a result, not as a failure.

1.6 Contributions

This thesis sets out to establish the following. This list is the intended contribution, not a report of measured outcomes. The experiments that would support it are specified in Chapter 5 and, once flown, belong in the results chapters.

1. **A controlled comparison.** To the best of our knowledge, the first evaluation of reactive, predictive and hybrid compensation as switch settings of one onboard controller, on the same airframe, the same trajectories and the same formations, with one definition of the residual for every flight, including those of strategies that do not use it, and with null flights, in the second phase, that establish the measurement floor. "Controlled" means more than identical experiments here: the reactive, the predictive and the hybrid strategy are switch settings of one controller with one shared set of gains, filters and physical parameters, so that a difference between them is attributable to the terms the switches add. The gains are frozen before the comparison flights, so a later difference is not mid-campaign retuning.

2. **A quantification of what the summed per-neighbour approximation costs** when a model trained on two vehicles meets three, in the second phase. This measures a penalty whose *existence* is already established in the literature [14, 4]; the contribution is its magnitude for a deployed model, not the finding that it exists.

3. **An engineering account of cost**: sensing prerequisites, onboard computation, training-data requirements and tuning effort, reported alongside performance so that the comparison supports a decision rather than only a ranking.

Supporting these, but explicitly *not* a scientific contribution, is the apparatus that makes them feasible: the frozen scenario library, the residual measurement path, the onboard learned model with fixed weights, and an offline training pipeline. It is described in Chapter 5 and offered for reuse.

What this thesis does not claim. No new control method is proposed: every strategy follows published work, and the contribution is the comparison. The residual model is not offered as an improvement on published architectures. Results concern a single airframe and a single class of disturbance; generalisation beyond either is discussed rather than demonstrated. No claim rests on simulation. And no strategy is claimed to be best in general, the useful outcome is conditional, which is RQ2.

A negative result would also be a contribution. If the strategies do not separate, the conclusion is that the choice of compensation family matters less than whether compensation is present at all, and the engineering question becomes one of cost. The experimental design is built so that this outcome is distinguishable from a failed experiment. The null conditions and the shared, frozen gains serve that purpose.

Provenance, not pending work. "To the best of our knowledge" is supported by two audits: an arXiv keyword sweep and a cross-database sweep including a citation-graph scan. Both found adjacent comparisons, cited in §2.8; the closest, Hsieh et al. (2025), is named in the gap statement, and none compared the families as switch settings of one onboard controller. Neither audit is exhaustive.

1.7 Scope

The scope is deliberately narrow, because the contribution is a controlled comparison and every degree of freedom left open is an alternative explanation for an observed difference. Table 1.1 lists what is in scope and what is not.

Table 1.1: Scope of this thesis.

In scope	Out of scope
Inter-vehicle aerodynamic interaction, principally downwash	Wind, gusts, and other external disturbances
Teams of two and three vehicles	Large swarms
Homogeneous vehicles	Heterogeneous teams
Tracking of pre-planned, collision-free formations	Interaction-aware planning; collision avoidance
Onboard inference, offline training	Online or on-board learning
The force residual, measured and predicted; the moment residual, measured	A learned model of the moment residual

1.8 Outline

Chapter 2 surveys what the disturbance is, how controllers measure or predict it, where a prediction enters the law, and why published results do not compose into one comparison.

Chapter 3 defines the residual this study measures and records, with one definition for every strategy. The chapter gives both routes in one notation and states the assumptions, with where each may fail.

Chapter 4 describes Strategies 0–4 (the uncompensated baseline, INDI, geometric+NN, INDI+NN and FBL+NN), and the point at which each injects its knowledge of the interaction, separating in each case the law that is flown from the published method it follows.

Chapter 5 describes the platform, the frozen scenario library, the measurement chain and the two-phase protocol, and states the threats to validity the design is built against, including those it does not eliminate.

The **two-robot results chapter** reports the results of the first, two-vehicle phase and the **multi-robot results chapter** those of the second, three-vehicle phase. The **discussion chapter** interprets them against the research questions and the stated assumptions. The **conclusion chapter** answers the research questions and lists future work.

Chapter 2

Background and Related Work

Multirotors flying in close proximity disturb one another aerodynamically. The disturbance is large enough to destabilise a vehicle, and conventional practice avoids it by keeping the vehicles far apart: a Crazyflie 2.0 measures 9 cm rotor-to-rotor, and when the interaction between vehicles is left unmodelled, planners have adopted conservative separations, Shi et al. [14] cite 60 cm from earlier planning work as one such example. Removing that margin is what makes dense formation flight possible, and doing so requires a controller that accounts for the interaction rather than avoiding it.

This chapter surveys how that has been attempted. It is organised as an argument rather than a catalogue. Section 2.2 establishes what the disturbance is. Sections 2.3 and 2.4 cover online measurement and offline prediction. Section 2.5 treats where a prediction enters the control law; Section 2.6 treats learning a corrective action without an explicit force estimate; Section 2.7 treats combining measurement and prediction. Section 2.8 examines how these methods are evaluated. The field's reporting conventions make the individual results impossible to compose into a comparison.

Families this thesis does not implement — direct action learning (Section 2.6) and residuals inside a predictive horizon (Section 2.5) — are described briefly. Chapter 4 states where this thesis's compared strategies end.

2.1 Multirotor dynamics and baseline control

Each vehicle i in a team is modelled as a rigid body,

$$\dot{p}_i = v_i, \qquad m\,\dot{v}_i = -mge_3 + f_i R_i e_3 + f_{\mathrm{res},i}, \tag{2.1}$$

$$\dot{R}_i = R_i \hat{\omega}_i, \qquad J\dot{\omega}_i = -\omega_i \times J\omega_i + \tau_i + \tau_{\mathrm{res},i}, \tag{2.2}$$

in a world frame with e_3 pointing up, where f_i and τ_i are the collective thrust and body torque produced by the rotors. The terms $f_{\mathrm{res},i}$ and $\tau_{\mathrm{res},i}$ collect every force and moment the nominal model omits. This chapter is concerned with the case in which they are dominated by the aerodynamic interaction with neighbouring vehicles. The formulation is developed in Chapter 3.

Rotations and the attitude error. The attitude R_i is a rotation matrix: an element of the special orthogonal group $SO(3)$, the 3×3 matrices with $R^\top R = I$ and $\det R = 1$, which maps vectors from the body frame to the world frame. Its third column $R_i e_3$ is therefore the direction of the thrust in the world, which is why the thrust appears as $f_i R_i e_3$ in (2.1). Position and attitude together form the pose, an element of $SE(3)$. The

hat map $\hat{\omega}$ in (2.2) turns a vector into the skew-symmetric matrix with $\hat{\omega}x = \omega \times x$; its inverse is the vee map $(\cdot)^\vee$. Working with R directly, rather than with three Euler angles, avoids the singular configurations at which a three-angle description loses a degree of freedom [9]. A geometric controller measures how far R is from a desired attitude R_d with the error function $\Psi(R, R_d) = \frac{1}{2}\operatorname{tr}(I - R_d^\top R)$ and the attitude error vector $e_R = \frac{1}{2}(R_d^\top R - R^\top R_d)^\vee$ [9]. For two attitudes that differ by a rotation of angle θ about some axis, $\Psi = 1 - \cos\theta$: zero when they coincide, one at 90 ° and two at 180 °. Chapter 4 uses both.

The baseline against which interaction-aware methods are measured is a geometric tracking controller on $SE(3)$, which tracks a position trajectory and a single heading direction coordinate-free, avoiding the representation singularities of a local attitude parameterisation [10]. Its guarantees are regional rather than global: exponential stability for initial attitude errors below 90 °, and almost-global exponential attractiveness below 180 °. It is the controller three of the strategies compared in this thesis build upon, and the one used uncompensated as a reference condition. The attitude error function is that work's, verified equation-by-equation against its companion arXiv paper, of which version 4 is held [9]; the torque law implemented here is a *reduced* form of it, and Chapter 4 states exactly which terms differ.

A second property of the quadrotor is used throughout: differential flatness. Position and yaw, together with their derivatives, determine the full state and the inputs, which allows a feedforward term to be computed directly from a planned trajectory. Tal and Karaman [17] exploit this to track position and yaw derivatives up to fourth order, using flatness-derived feedforward for angular rate and angular acceleration in order to follow jerk and snap. The chain follows from (2.1) and (2.2) with the residual set to zero. The desired acceleration fixes the thrust vector $fRe_3 = m(\ddot{p} + ge_3)$, hence the collective thrust and the body z-axis; the yaw angle then fixes the remaining rotation about that axis, so that R is determined. Differentiating once more, the jerk fixes the angular velocity, and the snap fixes the angular acceleration and with it, through (2.2), the torque [17, 7]. Position and yaw are therefore called the *flat outputs*. Flatness recurs in Section 2.5 as a structure that a learned correction can either preserve or destroy.

2.2 Aerodynamic interaction in multirotor teams

The dominant interaction in a vertically stacked formation is the downwash of the upper vehicle acting on the lower one. Two properties make it difficult to handle. It is strongly nonlinear, growing sharply as the vehicles close and decaying quickly with lateral offset; and it is asymmetric, since the vehicle above is affected far less than the one below. On a rig with two mounted Crazyflies, the lower vehicle's thrust drops to 65 % of its hover value at the closest vertical separation they report, four centre-to-motor distances (about 13 cm), while the upper one retains 90 %; laterally the lower vehicle's thrust is unaltered from an offset of about three centre-to-motor distances, whereas vertically the effect persists to about nineteen [8]. Measurements on a load stand likewise find a downwash that stays concentrated beneath the vehicle that produces it, with negligible influence on neighbours outside it apart from small, chaotic lateral forces [4].

Recent experimental work characterises the effect directly rather than inferring it from tracking error. Kiran et al. [8] measure forces *and* torques with load cells on mounted Crazyflies whose roll and yaw are locked and whose motors are driven at a constant hover voltage, and use particle image velocimetry in a light sheet through the centre of the front two rotors to resolve the wake, for a single quadrotor and for a pair; the dynamic cases

traverse one vehicle on a rail rather than flying it. Gielis et al. [4] measure six-degree-of-freedom exogenous forces on a 700 g multirotor held on a purpose-built load stand at hover thrust, while up to three further vehicles fly freely around it, and report strong nonlinearities in how those interactions aggregate. The vehicle experiencing the disturbance is therefore constrained rather than in free flight. Both measure the disturbance as a transducer load rather than inferring it from a flight controller's residual, actuation is still present in each, at hover thrust, which makes them, *in our reading*, the natural ground truth against which a learned model should be checked. Neither paper makes that claim. Gielis et al. evaluate their own learned aggregation models against the stand; neither paper evaluates a model learned from flight-controller residuals.

Simplified physical models exist. Bauersfeld et al. [1] analyse roughly 16 h of hover data across six platforms spanning 0.23–6.3 kg, and show that once the individual propeller wakes have merged the mean flow is well approximated by a turbulent jet — beyond about 2.5 motor-to-motor distances l below the vehicle, a length scale the abstract renders as "drone-diameters". That gives a computationally light alternative to computational fluid dynamics. What it estimates is the *time-averaged magnitude* of the induced flow *in hover*, not an instantaneous six-degree-of-freedom wrench on a follower. For the near field the authors report no closed-form first-principles model and point to computational fluid dynamics instead. Formation flight operates in exactly that near field — our observation, not theirs — which is the practical argument for learning the residual from data rather than deriving it. The model is nonetheless enough for a first-principles predictive compensator in the far field: used as a feedforward correction of the throttle while one vehicle passes 1 m and 2 m below another, it lowered the height RMSE by a factor of about three to five [1].

2.3 Reactive compensation: measuring the residual online

The first family estimates the residual from its effect and requires no prior model of the interaction. Incremental nonlinear dynamic inversion (INDI) compares measured acceleration with the acceleration the nominal model predicts for the actuation actually applied, and attributes the difference to unmodelled forces, which it then cancels incrementally.[1]

The mechanism can be stated in one line. If the actuation currently applied produces, according to the nominal model, an acceleration a_{model}, and the accelerometer reports a_{meas}, the difference is whatever the model omits. INDI does not compute the command from the model alone but increments the current one: the new command is the current actuation plus the actuation needed to move the measured acceleration to the desired one, so that the unmodelled part is carried along inside the measurement rather than being modelled [17, 15]. The measured acceleration is noisy and must be low-pass filtered, which delays it. The estimate of the current actuation must then be passed through an identical filter, so that both terms incur the same phase lag [17, 15]; otherwise their difference contains a term produced by the unequal delay rather than by a force.

Smeur et al. [15] cascade an inner angular-acceleration INDI loop with an outer linear-acceleration INDI loop on a single Parrot Bebop running Paparazzi at 512 Hz; the position and velocity loops feeding the acceleration reference remain proportional-derivative rather

[1] "Reactive" is used in this thesis for methods that *measure* the residual from its effect, whether by INDI or by another online estimator such as an $\mathcal{L}_1$-adaptive module [6], an adaptive scheme that estimates the disturbance from the mismatch between a predicted and a measured state. Part of the model predictive control (MPC) literature uses the word differently, for any controller that compensates only the disturbance at the current time step, including feedforward of a learned model [2]; in this thesis such controllers are predictive.

than a third incremental stage. They demonstrate disturbance rejection indoors by flying in and out of a 10 m/s open-jet exhaust every 14 s, and in that same experiment the motion-capture position reached the vehicle at only 4 Hz. A separate outdoor experiment, taking off and holding position on a consumer GNSS module in roughly 5.1 m/s average wind, likewise shows that the method does not depend on frequent, precise position updates. Tal and Karaman [17] use INDI for robust tracking of linear and angular accelerations in aggressive flight on a single custom 609 g vehicle, and state the property that defines the family: no parametric model of the aerodynamic effects is required. That claim is narrower than it sounds. Mass, inertia, the motor time constant, control effectiveness and the throttle map are still identified. What is dropped is the drag model.

The same work also exposes the family's cost. Tracking snap requires direct control of body torque, which those authors achieve using closed-loop motor speed control based on optical encoders on their own vehicle's motors [17]; encoders are what *that* snap-and-torque path required, not a general prerequisite for INDI. Reconstructing the applied wrench well enough to difference it against measurement does, however, generally require some form of rotor-speed feedback, Smeur et al. likewise use motor-speed measurement [15], which is additional hardware or firmware support that must be fitted, calibrated and maintained on every vehicle. On a Crazyflie 2.1, an INDI controller that reconstructed the thrust from the motor commands instead of measured rotor speeds tracked worse than the uncompensated geometric controller it was meant to improve [3]. A second limitation is structural rather than practical: the estimate is delayed by the sensing and filtering chain, and it exists only once the disturbance has already perturbed the vehicle. A reactive controller cannot, even in principle, act before the disturbance arrives. Gielis et al. make the same point as a motivation for prediction, that reactive controllers cannot reject the interaction without first observing it [4]. Shankar et al. note that a vehicle docking beneath a leader meets a downwash that a disturbance observer struggles to estimate because its convergence time is finite [12], and Neural-Swarm2 reports that a simple online estimate-and-compensate scheme quickly crashed the vehicle in their experiments [13]. The last is a single remark without the estimator, its filter or data, and is evidence about that scheme rather than about INDI.

That the sensing requirement is not fundamental has since been shown. Cobo-Briesewitz et al. [3] demonstrate that a neural network can generate smooth approximations of INDI's residual estimate without rotor-speed measurement, which is discussed in Section 2.7.

2.4 Predictive compensation: learning the residual from relative state

The second family predicts the residual before it acts, from the relative states of the neighbours, using a model learned offline from flight data. Because the interaction is a function of the team's relative configuration, a model with the right inputs can act ahead of the disturbance rather than after it.

Shi et al. [14] combine a nominal dynamics model with a regularised permutation-invariant deep neural network that learns multi-vehicle interaction, and use the learned model as a feed-forward term in a decentralised nonlinear tracking position controller; the design leaves the attitude controller open, and the experiments use the nonlinear attitude tracking controller of Lee et al. The learned force itself is the vertical component $f_{a,z}$, evaluated onboard the STM32. Training separate networks on data from two, three and four vehicles respectively, they report worst-case height error reduced by a factor of two or better, and up to four times smaller in the most favourable case. In a two-vehicle

swapping task the baseline reaches 9.4 cm; the network trained on *four* vehicles brings this to 2.4 cm, while the network trained on two vehicles reaches 2.7 cm. The swap experiments use vertical neighbour spacings of 0.20–0.25 m, with 25 cm stated explicitly for the three-vehicle case. Generalisation is directional rather than universal: networks trained on three or four vehicles improved five-vehicle flight, whereas a network trained only on two vehicles did *not* generalise, performing worse than the baseline on three- and four-vehicle tests.

Neural-Swarm2 [13] extends this to heterogeneous teams using heterogeneous deep sets, whose permutation-invariant architecture accepts any number of neighbours. That is a structural property rather than an accuracy claim: the networks are trained on at most three vehicles (at most two neighbours), and the paper describes prediction at sixteen robots as relatively less accurate. Two design choices are worth noting. Spectral normalisation is applied: it bounds the Lipschitz constant of a network, the largest factor by which its output can change relative to a change in its input, by constraining the spectral norm of each weight matrix [14]. The abstract describes this as yielding stability and generalisation guarantees; in the body this rests on Lipschitz bounds together with empirical generalisation and a bounded-residual assumption used in the proofs, rather than a formal guarantee on unseen data. The learned residual is used in interaction-aware motion planning as well as in tracking control. The abstract reports up to a threefold reduction in worst-case tracking error against a baseline nonlinear tracking controller with the interaction force set to zero; the introduction and its first figure report a heterogeneous team of sixteen robots flown at a minimum vertical distance of 24 cm. We do not assume the two describe the same experiment: on the sixteen-robot flight itself the body reports the maximum z-error of one small vehicle with upgraded motors, in one of the vertical rings, falling from 15 cm to 10 cm, a factor of about 1.5.

Subsequent work has focused on making these models cheaper to obtain rather than larger. Smith et al. [16] exploit the rotational symmetry of the problem with an $SO(2)$-equivariant model, and report that five minutes of real-world flight data attains a lower validation error than learning baselines trained on more than fifteen — a data-efficiency result on prediction accuracy rather than a closed-loop tracking one. Deployed online within an optimal feedback (LQR) controller on two identical vehicles, the model reduces three-dimensional tracking error by 36 % and, as the abstract states, vertical tracking error by 56 % on average.[2] On a lateral transect flown beneath the hovering vehicle they report mean three-dimensional position error falling from 0.154 m to 0.098 m, close to 36 %, with vertical improvements of 55 % in position and 49 % in velocity. The platform is not a Crazyflie but a custom quadrotor of roughly 0.26 m span and 0.7 kg, tracked by OptiTrack and controlled from a Raspberry Pi at 50 Hz with a 45 Hz network update. The closest *training* stage is about 0.5 m — some two body-lengths — while the evaluation manoeuvres are flown at 0.6–0.8 m, together with a lemniscate flown beneath the leader hovering at a height of 2.5 m ($z = -2.5\,m$ in their downward-pointing frame). The authors attribute the efficiency to exploiting the problem's latent geometry rather than learning it from data.

The superposition assumption The deep-sets architectures above aggregate per-neighbour terms by summation, which commits them to a form of superposition. The literature is explicit that this is an approximation. Shi et al. [14] observe that with more than two vehicles the aerodynamic effect is not a simple superposition of each pair. Gielis

[2] The paper is internally inconsistent on whether the 36 % figure is three-dimensional or lateral: we follow the abstract and the translation-result body, which give three-dimensional and vertical. Its contributions list instead calls the 36 % figure lateral. The 56 % figure appears in the abstract and the contributions only; the body reports 55 % for the translation experiment and about 51 % for the lemniscate.

et al. [4] make the same point as their motivation, an accurate model of the downwash from one vehicle is unlikely to suffice for predicting the aggregate from several. Their abstract frames this as modelling the formation as a graph; the implementation learns the aggregation function with a sum-pooling deep-sets predictor, contrasted against a linear-summation baseline. The abstract announces an exploration of how the models generalise when a fourth vehicle is added; the largest reported case is three neighbours around the stand, four vehicles in total, and the conclusion cautions that the deep-sets model may be incapable of generalising beyond three neighbours, so we do not treat generalisation to larger formations as a demonstrated result.

Their measurements refine the picture rather than simply refuting summation. They report that interactions fall into three regimes depending on the formation and its location in the state space: a chaotic, noise-dominated regime; a linear regime *well estimated by linear summation*; and a nonlinear regime requiring a nonlinear model. Superposition is therefore not uniformly wrong but regime-dependent, which makes the practical question one of how much accuracy is lost, and where. This is their own load-stand result, obtained independently of Shi et al.'s free-flight observation cited above. Both readings are in use: Hsieh et al. cite the same measurements in support of approximating the aggregate force on a vehicle as a pairwise sum [6].

Summed architectures nevertheless remain in wide use, because a controller does not require an exact model, only one whose remaining residual is smaller than the one it removes. What is not reported is the size of the penalty that the simplification carries for a deployed model, which is the question this thesis takes up in the multi-robot results chapter.

2.5 Injecting the prediction: the controller matters

A predicted residual is inert until it enters a control law, and the manner of entry differs by controller family. Treating "predictive compensation" as a single method would obscure differences that are as large as those between the families themselves.

Feedforward into a geometric controller. The simplest injection adds the predicted force to the desired force of the baseline controller, leaving its structure unchanged. This is the form used with the permutation-invariant models of Section 2.4, whose baseline is a nonlinear tracking controller [14, 13]. The closest published instance with a geometric controller on $SE(3)$ is a configuration in the hardware study of Hsieh et al. [7]: on two Crazyflie 2.1 vehicles, feeding a learned residual forward into an $SE(3)$ controller reduced the tracking RMSE by 43.7 % against the nominal $SE(3)$ controller, with the controller computed off-board.

Feedback linearisation with a flatness-preserving residual. A learned correction added to a nominal model can destroy differential flatness, and with it the applicability of flatness-based planning and control. Yang et al. [18] show that residuals with a lower-triangular structure preserve both the local flatness of the system and its original flat output for systems whose nominal model admits a pure-feedback form, and give a constructive procedure for recovering the diffeomorphism of the augmented system. The residual there is a *structured correction inside the state derivative* rather than a free six-degree-of-freedom wrench — in their worked example it appears only in the velocity equation. Their validation is in simulation on a planar, two-dimensional quadrotor; there is no hardware, no three-dimensional vehicle and no multi-robot downwash experiment, and they list a

physical quadrotor as future work. Hsieh et al. [7] apply the idea to tight formation flight, learning a physics-informed residual that keeps the joint multi-quadrotor system differentially flat and using the preserved structure to design a feedback-linearising controller that cancels the disturbance by feedforward. Their abstract reports a 31 % reduction in average tracking error against nominal baselines; the body does not restate that aggregate, reporting instead, on hardware, 24 % RMSE and 30 % maximum-z improvement for reduced-order-model feedback linearisation (ROM-FBL) over nominal feedback linearisation (FBL), and a further 29 % RMSE and 43 % maximum-z improvement for learned FBL over ROM-FBL.

On hardware they match the approximately 5 cm RMSE and 10 cm maximum-z error *reported* by Chee et al. [2], rather than comparing the two methods on one rig, although their introduction describes the comparison as made in an identical hardware setup; the head-to-head comparison with nonlinear model predictive control (NMPC) is in their simulation study, where the flatness-based controller's maximum-z error can be worse than that of NMPC.

The 20-fold reduction in solve time (about 1 ms against 27 ms at equal residual) is a *simulation* figure. The abstract and the introduction state an order of magnitude less computation. Training uses 28 s of hardware data and 18 s in simulation.

Two qualifications matter for our purposes: the hardware experiments use two Crazyflie 2.1 quadrotors with thrust-upgrade bundles — very nearly the platform used in this thesis — and the controller runs *off-board*, on an Intel i5 laptop streaming commands over a Crazyradio link at 400 Hz, rather than on the vehicle. Their 5 ms loop is therefore an abstract-level, off-board figure, and the claim it supports is that the method is cheap enough to be worth porting, not that it has been shown to run onboard.

On hardware they also compensate residual *forces* only, commanding thrust and angular velocity, and delegating residual torques to the vehicle's own rate controller, the same scoping decision taken in Chapter 3. Their simulation study does still model residual torques.

Inside a model predictive control horizon. The residual can instead form part of the prediction model that an optimiser reasons over, so that the controller plans against a disturbance it expects to meet rather than cancelling the one present now. Chee et al. [2] embed a learned component inside the prediction constraint of a nonlinear MPC (NMPC); Li et al. [11] inject a predicted downwash-force sequence into the optimiser's prediction model. Hsieh et al. [6] add to the MPC of Chee et al. an $\mathcal{L}_1$-adaptive module that estimates the residual acceleration online, and fly three Crazyflies in two stacked formations with no compensation, the online estimate only, the learned prediction only and both. The combination performs best, and for the middle vehicle the online estimate alone outperforms the learned prediction alone; the authors also note that the estimate without a downwash model responds more slowly when a vehicle enters or leaves the wake. Chee et al. and Hsieh et al. fly Crazyflies but run the optimisation off-board. Li et al. run their controller on a TX2 NX computer carried by the vehicle, which is not of the Crazyflie class. None of the three runs its optimiser onboard a Crazyflie-class vehicle. This injection point is not among the strategies evaluated here — its onboard feasibility on a Crazyflie-class platform is itself open — but it is retained in this survey because its evaluation conditions form part of the argument in Section 2.8.

Inside an optimal feedback controller for a docking task. A further instance is worth separating because its task differs in kind. Shankar et al. [12] embed a learned

lumped downwash force online in an infinite-horizon LQR acting on a feedforward-linearised follower, beneath an inner attitude loop, and use it for vertical docking: a leader–follower manoeuvre in which the vehicles close to within one or two body-lengths, so that the disturbance is not incidental to the task but constitutive of it. The platforms are two custom quadrotors of roughly 0.27 m diagonal and 0.7 kg, not the Crazyflie class, and the free-flight dock is performed at a vertical offset of about 0.36 m — close to one body-length — with a curriculum descending to around 0.5 m. The model itself is the $SO(2)$-equivariant one of their prior work, trained offline from about five minutes of data and queried online on an onboard Raspberry Pi, with pose from motion capture; the evaluation is hardware only. Reported accuracy depends on the manoeuvre: the abstract gives terminal error below 0.06 m, the body reports terminal vertical error typically below 0.1 m for a hovering leader, and for a moving leader below 0.05 m with the model against more than 0.2 m without it. Physical docks succeeded in eight of ten attempts against a hovering leader at 0.36 m; against a leader moving at 0.25 m/s success was limited, which the authors attribute to the follower's oscillation preventing the gripper from closing, and to the gripper's actuation latency. It is the closest published setting to the tightest scenarios of Chapter 5, and the regime in which — on our reading — compensation ceases to be an accuracy improvement and becomes a precondition for the manoeuvre.

2.6 Learning the compensation directly

A third stance declines to model the force at all and learns the corrective action instead. Zhang et al. [19] train a residual reinforcement-learning module on top of a cascaded controller, compensating the disturbance and thrust loss caused by downwash without forming an explicit force estimate at any point. Two properties distinguish the stance sharply from Section 2.4. The policy observes only tracking error, the baseline controller's command and its own previous action — no neighbour state and no inter-vehicle communication — which removes a bandwidth requirement and a failure mode, at the cost of being unable to anticipate a neighbour it cannot perceive. And giving up the explicit force estimate removes the predicted-against-measured comparison that every other learned method here is evaluated by, which makes failures correspondingly harder to attribute.

This family is not among the strategies evaluated here, for that second reason as much as for scope: the evaluation built in Chapter 5 rests on the residual being a measurable quantity, and a method that never forms one is assessed on tracking performance alone. It remains the most natural direction in which to extend the comparison.

2.7 Hybrid reactive and predictive compensation

Measurement and prediction are complementary in principle. Prediction is available before the disturbance acts but is valid only over the configurations its training data covered; measurement is general but delayed. Combining them is therefore natural. It has been done with INDI for a single vehicle, described below, and, inside an off-board model predictive controller with an $\mathcal{L}_1$ estimate, for a team of three (Section 2.5) [6].

Cobo-Briesewitz et al. [3] propose two methods. In the first, LINDI, a neural network replaces INDI's residual estimate entirely, trained on smoothed data obtained by spline fitting. In the second, NA-INDI, the total residual is written $f_a = f_{a,\mathrm{NN}} + f_{a,\mathrm{INDI}}$: the network predicts a component and the incremental term reasons about the remaining mismatch. Both are evaluated on a single quadrotor, with and without a slung payload, on figure-of-eight and circular trajectories, neither of which was in the training set.

The headline result concerns sensing rather than the hybrid: a network can generate smooth approximations of INDI's output without requiring rotor-speed sensors at LINDI's runtime, which removes the hardware prerequisite identified in Section 2.3 from that method's deployment — though rotor speeds are still needed to produce the training labels, and NA-INDI retains an incremental branch that uses them. Without a payload, residual compensation of any kind reduces tracking error by around 40 % relative to the geometric baseline; with a payload the reductions are smaller, in the range of 19–24 %. The hybrid's own advantage is real but modest: without a payload it gives the best performance, though the authors describe the margin as small, and with a payload the authors describe it as remaining *similar* to INDI alone. They also observe a slight drop in performance on the circular trajectory — for LINDI, in the no-payload case — attributing it to that trajectory being out of distribution relative to the training data, a direct instance of the limitation that defines the predictive family.

Two features of this setting bear on whether the result transfers. The residual there comes from the vehicle itself — the authors list blade flapping, drag and ground forces among the sources of such residuals — and, in the payload experiments, from a slung payload, and it is predicted from the vehicle's own state and motor commands. In a multi-robot team the residual is produced by other vehicles and is a function of a measurable relative state, which gives the predictive component an input the single-vehicle setting does not provide. Whether that turns a small margin into a decisive one has been examined on a team only inside an off-board MPC with an $\mathcal{L}_1$ estimate [6], not for an INDI-based hybrid. This thesis flies such a hybrid on a team, in the variant in which the measured and the predicted term are added rather than partitioned (Chapter 4).

2.8 How these methods are evaluated

The preceding sections describe a field with strong results. Shi et al. [14] report up to a fourfold reduction in worst-case height error. Neural-Swarm2 [13] reports up to a threefold reduction in worst-case tracking error against a zero-interaction baseline in one experiment; the sixteen-robot flight is a separate report. Smith et al. [16] report 36 % three-dimensional and, in the abstract, 56 % vertical tracking improvements. Hsieh et al. [7] state 31 % against nominal baselines in the abstract; the hardware body gives other figures. Cobo-Briesewitz et al. [3] report roughly 40 % for residual compensation without a payload. Taken individually, each result is credible.

They cannot, however, be composed. Table 2.1 lists the conditions under which they were obtained.

A 36 % improvement measured on two vehicles in close proximity and a 31 % improvement measured in a tight formation are not the same quantity: they differ in what was flown, on what airframe, at what separation, against what baseline, and under what metric. Neither can be subtracted from the other, and their ordering carries no information about which method would prevail if both were flown under the same conditions. The difficulty compounds where the baselines themselves differ, since an improvement relative to a weaker baseline is easier to obtain.

The practice behind the rows has common features. Most results are within-paper ablations: one controller with the learned term switched on and off, so that each improvement is relative to that paper's own nominal controller [14, 13, 2, 11, 7]. Metrics converge on a root-mean-square error and a maximum vertical error, but over windows that differ — a whole flight, a crossing, a terminal approach. Repetitions are few, between three and twenty per condition [11, 14, 7, 12, 3]. Separations are mostly reported as commanded;

Chee et al. report the achieved separation as well, which in their tightest case differed markedly from the commanded one [2]. Data are split for validation in different ways: by contiguous pieces of flight in Neural-Swarm2 [13], by shuffled samples in Li et al. [11], by held-out trajectories or separations elsewhere [3, 7]. Simulation, a processor in the loop with a simulator, and flight are different kinds of evidence [5, 18], as are onboard and off-board execution when computational cost is claimed. And the baselines are rarely tuned for themselves; where tuning for fairness is explicit, it is the exception [5].

This is not a criticism of any individual work. Each is internally valid, and there is no convention in the field that any of them violated. It is a limitation of the literature taken as a whole: the results are not designed to compose, and no amount of careful reading assembles them into a ranking. A ranking requires an experiment built for that purpose. That is what distinguishes a comparison from a survey.

Comparisons that do exist The field does conduct systematic comparisons, and a few bound what remains open.

Gu et al. [5] benchmark four online-learning uncertainty estimators — extreme learning machines, radial-basis-function networks, echo state networks and Gaussian processes — inside a single modular framework in which a nominal controller is paired with a swappable residual-learning module. They treat the uncertainty as a lumped additive wrench (f_a, τ_a) recovered by inverting the discrete Newton–Euler dynamics — analogous to the residual acceleration used here, though not identical to it a priori. They evaluate estimation accuracy, tracking performance and computational cost, and reach the kind of conclusion a comparison should: not a winner, but a set of trade-offs, with Gaussian processes most accurate yet limited by cubic training complexity, the lightweight parametric models fast but sensitive to initialisation, and echo state networks the most balanced.

Two things separate it from the present work. Its subject is a *single* quadrotor under generic uncertainty, and all four methods learn *online*; it therefore compares estimators *within* one family rather than the families of Sections 2.3 to 2.7. And its evaluation is in simulation throughout: the reported hardware component is the learning modules executing on an embedded computer in the loop with a simulator, which measures latency rather than flight, and the authors list validation on a real quadrotor as future work.

Its approach to fairness is nonetheless directly relevant. They tune the parametric models — the extreme learning machine, the radial-basis-function network and the echo state network — to their own optima by Bayesian optimisation under otherwise consistent conditions, so that each operates near its best achievable performance. This thesis answers the same objection — that an observed difference might reflect tuning effort rather than method — differently: its reactive, predictive and hybrid strategies are switch settings of one controller with one shared gain set, frozen before the comparison and the training flights. Section 4.7 states what that does and does not guarantee.

Hsieh et al. [6] come closest: they compare an online estimate, a learned prediction and both on three vehicles inside one off-board model predictive controller (Section 2.5). The estimator and the controller are one off-board optimiser, so the families are not compared as settings of one onboard controller.

Together these delimit the gap precisely. It is not that close-proximity control is uncompared, nor that the field lacks the methodology. It is that the specific families defined by *how they obtain information about the interaction force*, measure it, predict it, or both, have not been placed under common conditions as switch settings of one onboard controller; the closest study places the three information sources under common conditions inside one off-board model predictive controller [6].

Such an experiment has requirements that follow directly from the failure modes above: a single airframe; identical trajectories and formations across all methods; gains frozen before data collection, so that a difference cannot be a tuning artefact; a single definition of the residual, measured identically for every controller including those that do not use it; and a null condition in which no interaction is present, to establish the measurement floor and so distinguish a real effect from noise. Chapter 5 describes the resulting design.

2.9 Summary and gap

A controller can obtain information about the interaction force in two ways. It can measure the residual online, which requires no prior model and generalises to disturbances never seen, but is delayed and, in its established form, depends on rotor-speed sensing. Or it can predict the residual from the relative states of its neighbours using a model learned offline, which acts ahead of the disturbance but is valid only where the training data reached. A third approach learns the corrective action directly, forgoing an explicit force estimate and, in at least one instance, inter-vehicle communication as well. Representative methods exist for each, and each is individually validated.

Three gaps follow from this survey.

First, and principally, to the best of our knowledge no study has compared rotor-speed-based reactive compensation, learned per-neighbour predictive compensation and their combination, with the measured and the predicted term added, as switch settings of one onboard controller on one airframe, across a fixed library of interaction regimes, with one definition of the residual for every flight. Systematic comparisons in adjacent territory exist — of online-learning estimators on a single vehicle, and one that places an online estimate, a learned prediction and both inside one off-board model predictive controller on three vehicles [6] (Section 2.8) — but none compares the families as switch settings of one onboard controller. The individual results are strong; what the field lacks is not strong results but comparable ones, and its reporting conventions — improvements against each work's own baseline, on its own platform, at its own separations — make the comparison impossible to assemble after the fact. This thesis compares the uncompensated baseline, reactive compensation (INDI), predictive compensation (geometric+NN) and their hybrid (INDI+NN), and FBL+NN if it is implemented.

Second, the hybrid of measurement and prediction has been demonstrated with INDI only for a single vehicle, where the residual is self-induced and its advantage was described as small without a payload and as leaving performance similar to the reactive method with one [3]. On a team it has been flown inside an off-board model predictive controller with an $\mathcal{L}_1$ estimate, where the combination performed best [6]. An INDI-based hybrid on a team, in which the residual is larger and is a function of a measurable relative state, remains untested; this thesis evaluates one on a team, with the measured and the predicted term added.

Third, the summation used to aggregate per-neighbour predictions is known to be an approximation [14, 4], yet the magnitude of the penalty it carries for a deployed model, evaluated on teams larger than those it was trained on, is not reported. A summed model trained on two-vehicle data has been flown on three vehicles [6], but the penalty was not isolated.

Provenance, not pending work. Supported by two audits: an arXiv keyword sweep, and a cross-database sweep over OpenAlex, Crossref and DBLP including a citation-graph scan of 157 works citing the Neural-Swarm line. Both found adjacent comparisons; none compared the families as switch settings of one onboard controller, and the closest (Hsieh et al.) is named in the gap statement. Neither is exhaustive; keep the "to the best of our knowledge" phrasing.

Chapter 3 formalises the residual wrench these methods estimate or predict, and Chapter 4 describes the strategies compared in this thesis and the point at which each injects its knowledge of the interaction into the control law.

Table 2.1: Evaluation conditions across the interaction-force literature. Each result is reported against that work's own baseline; rows that share a platform and team size still differ in baseline controller, separation or reported metric.

Work	Team size	Separation	Reported quantity
Tal and Karaman [17]	1	n/a (no formation)	aggressive tracking
Smeur et al. [15]	1	n/a	jet / GNSS position error
Shi et al. [14]	2–4 train, 2–5 test	0.20–0.25 m vert.	max. $\|z\|$ error
Neural-Swarm2 [13]	16, heterogeneous	24 cm vert.	worst-case tracking error[†]
Smith et al. [16]	2, custom 0.7 kg	0.6–0.8 m eval. (train. to ≈0.5 m)	3D and vertical tracking; onboard Pi, mocap
Gielis et al. [4]	up to 4 total (rig sufferer + ≤ 3 flyers)	test rig, 700 g sufferer not in free flight	force-prediction accuracy
Yang et al. [18]	1, planar (sim. only)	n/a	simulation only
Hsieh et al. [7]	2, Crazyflie 2.1	0.3 m and 0.4 m vert.[‡]	RMSE and max. z error vs. nominal FBL/$SE(3)$; off-board
Chee et al. [2]	2, Crazyflie 2.1	commanded 0.2–0.4 m; tight case ≈0.08 m	RMSE / max. z vs. nominal MPC; off-board
Li et al. [11]	2, custom	neighbour-relative	NMPC with predicted force sequence; onboard TX2 NX + mocap
Hsieh et al. [6]	3, Crazyflie 2.1	V-stack and I-stack; 0.2 m and 0.3–0.4 m vert.	RMSE / max. z; estimate, prediction and both, in MPC; off-board
Shankar et al. [12]	2, custom (≈0.27 m)	dock ≈0.36 m	terminal vert. error / dock success; onboard Pi, mocap
Cobo-Briesewitz et al. [3]	1 (+ payload)	n/a	position tracking error
Zhang et al. [19]	2, heterogeneous	0.50 m vert. hover/circle; dock to 0.10 m, release at 0.05 m	position and attitude error

[†] The sixteen-robot flight and the threefold reduction are separate reports; on the sixteen-robot flight the body gives a factor of about 1.5. [‡] Hardware tracking comparisons are at commanded $\Delta z = 0.3\,m$ and 0.4 m; the 0.2–0.5 m range belongs to the simulation and prediction evaluation.

Chapter 3

System Modelling and Residual-Force Formulation

Chapter 2 surveyed families that measure a residual, predict one, plan over one, or learn a corrective action with no force estimate at all. Those families are comparable as *controllers*, on the same vehicles, trajectories and formations. They are not required to compute the same internal quantity.

This chapter defines the residual acceleration a_{res} that *this* study measures and records. It has one definition for every strategy: the controller computes it onboard when it uses it, and otherwise it is computed after the flight from the recorded signals in the same way. The strategies therefore refer to one quantity rather than to several differently-defined ones: Strategies 1 and 3 use the measured residual in all three axes, and the learned model of Strategies 2 and 3 predicts its vertical component. A predicted residual can therefore be checked against a measured one on the same flight. That is how a learned model is evaluated here.

The chapter also states the assumptions the formulation rests on, and where each one is expected to fail. Several are known to be imperfect, the summation in Section 3.4 most conspicuously, and stating them here is what allows the discussion chapter to interpret a negative result as evidence about an assumption rather than as an unexplained failure.

3.1 Notation and conventions

Vectors are expressed in a world frame that is *east-north-up*: the third basis vector e_3 points away from the ground and gravity acts along $-e_3$. This matches the convention used by the flight software and the motion-capture system, and is stated explicitly because the opposite convention is common in the quadrotor literature and the sign of the gravity term differs between them.

For vehicle i: $p_i \in \mathbb{R}^3$ is position, $v_i \in \mathbb{R}^3$ velocity, $R_i \in SO(3)$ the rotation from body to world, and $\omega_i \in \mathbb{R}^3$ the body angular velocity. The hat map $\hat{\cdot} : \mathbb{R}^3 \to \mathfrak{so}(3)$ is defined by $\hat{a}b = a \times b$, and the vee map $(\cdot)^\vee$ is its inverse, so that $(\hat{a})^\vee = a$. A hat over an angular velocity, as in $\hat{\omega}_i$, always denotes the hat map; a hat over an acceleration, as in $\hat{a}_{\text{res}}$, denotes a *prediction*. The vehicles are homogeneous, of mass m and diagonal inertia J. Rotor k spins at ϖ_k, in radians per second, and produces thrust $k_t \varpi_k^2$ along the body z-axis, with one thrust constant k_t for all four rotors.

3.2 Rigid-body model with a residual wrench

Each vehicle is modelled as a rigid body driven by a collective thrust f_i along its body z-axis and a body torque τ_i:

$$\dot{p}_i = v_i, \qquad m\,\dot{v}_i = -mge_3 + f_i R_i e_3 + f_{\mathrm{res},i}, \tag{3.1}$$

$$\dot{R}_i = R_i \hat{\omega}_i, \qquad J\dot{\omega}_i = -\omega_i \times J\omega_i + \tau_i + \tau_{\mathrm{res},i}. \tag{3.2}$$

The terms $f_{\mathrm{res},i}$ and $\tau_{\mathrm{res},i}$ form the *residual wrench*: by construction they collect every force and moment that (3.1)–(3.2) otherwise omits. This includes blade flapping, rotor drag, ground effect and errors in m, J and k_t, as well as the inter-vehicle aerodynamic interaction that is the subject of this thesis.

That the residual is a catch-all rather than a physical quantity has two consequences that matter for the experiments. First, a residual measured in isolated free flight is not zero; it is whatever the nominal model gets wrong about a single vehicle. Second, the interaction can only be isolated by *comparison*: the residual measured in formation is compared with the residual measured without a neighbour. The coplanar control scenarios in Chapter 5 exist for exactly this purpose.

Scope: the force is predicted, the moment only measured The learned model of this thesis predicts the *force* residual f_{res}, and only its vertical component. The moment residual τ_{res} is not predicted. The strategies that use the measurement, Strategies 1 and 3, correct the attitude loop with a measured moment term (Chapter 4); the others leave the moment to the attitude loop, which rejects it as a disturbance in the usual way. This is a scoping decision, not a claim that τ_{res} is negligible.

The decision has company. Neural-Swarm compensates the vertical interaction force [14], and the hardware controller of Hsieh et al. [7] likewise commands thrust and angular rates, leaving residual torque to the rate loop. Kiran et al. [8] and Gielis et al. [4] do measure both force and moment, but on a rig rather than as a flight residual.

We therefore do not learn τ_{res}. The cost is that, without the measured moment term, interaction torque is left to the attitude loop. The benefit is a single learned output that fits the onboard budget. The discussion chapter returns to what leaving the moment unpredicted may cost.

3.3 Measuring the residual

The residual can be recovered without any model of the interaction, by comparing what the vehicle actually accelerated with what the nominal model says it should have accelerated given the actuation actually applied. Rearranging (3.1),

$$\frac{f_{\mathrm{res},i}}{m} = \underbrace{\dot{v}_i}_{a_{\mathrm{meas}}} - \underbrace{\left(\frac{f_i}{m} R_i e_3 - ge_3\right)}_{a_{\mathrm{model}}}. \tag{3.3}$$

Both terms are available onboard. An accelerometer does not measure $\dot{v}_i$ directly: it measures the *specific force*, the acceleration of the body minus the acceleration of gravity, expressed in the body frame, $\tilde{a}_i = R_i^\top(\dot{v}_i + ge_3)$. A vehicle resting on the ground therefore reads $\tilde{a}_i = R_i^\top ge_3$, one g upwards, although it does not accelerate. Rotating the reading into the world frame and subtracting ge_3 gives $a_{\mathrm{meas}} = R_i\tilde{a}_i - ge_3 = \dot{v}_i$. The model acceleration requires the applied thrust, which is reconstructed from measured rotor speeds

as $f_i = \sum_{k=1}^{4} k_t \varpi_k^2$. The rotor speeds come from an optical rotor-speed sensor on each vehicle. Both accelerations, the rotated accelerometer reading and the model acceleration, pass identical second-order Butterworth low-pass filters at 30 Hz, sampled at 500 Hz, before they are subtracted.

Two illustrative magnitudes. The numbers in this paragraph are illustrative and are not measurements of this project. Consider a vehicle of $m = 0.041\,kg$ in hover, so that the true thrust equals $mg \approx 0.40\,N$ and $\dot{v}_i = 0$. If the identified thrust constant overstates the real thrust by 5 %, the model believes the rotors deliver $1.05\,mg$, so $a_{\text{model}} = +0.05\,g\,e_3$ and (3.3) returns $a_{\text{res}} \approx -0.49\,\text{m/s}^2$, a steady downward "force" that no neighbour produced. For comparison, Kiran et al. measured on a static rig that the lower of two vertically stacked Crazyflies, 13 cm apart and driven at a constant hover voltage, produced about 65 % of its nominal hover thrust [8]. If a loss of that size were not captured by the rotor-speed thrust model, it would appear as a residual of the order of $0.35\,g$, about $3.4\,\text{m/s}^2$. The two magnitudes differ by less than an order of magnitude, which is why model error must be bounded separately (Section 3.3.1).

When the measured residual is used (switch A, Chapter 4), the controller evaluates (3.3) at its own rate of 500 Hz. When it is not used, the controller does not compute it, and the same quantity is computed after the flight from the recorded accelerometer signal, attitude and rotor speeds, with the same constants and the same filters. The definition is therefore one for all strategies. This matters twice. The training data of the learned model are flown under the *uncompensated* controller, so their labels must exist when no INDI term is active; and comparing a predicted residual to a measured one on the same flight requires both quantities for every flight. That is how a learned model is evaluated.

3.3.1 What this measurement requires, and how it fails

Four properties of (3.3) shape the experimental design.

It requires rotor-speed sensing. Without ϖ_k the applied thrust is unknown and a_{model} cannot be formed. This is the hardware prerequisite identified in Section 2.3, and the same one that Cobo-Briesewitz et al. [3] remove by learning the residual instead of computing it. How much the quality of the thrust estimate matters is visible in their own results: on a Crazyflie 2.1, an INDI variant that reconstructed thrust from the motor command rather than from measured rotor speed tracked worse than the uncompensated geometric controller, which the authors read as INDI's dependence on accurate sensor measurements [3]. Without the rotor-speed sensor the controller does not form (3.3) at all, and the switches that use it have no effect; the residual terms then read *identically zero*. That failure is silent and is indistinguishable, in a log file, from a flight in which no interaction occurred. Verifying that the rotor speeds and a_{res} are non-zero is therefore the first check of every experimental session (Chapter 5).

It is delayed. a_{meas} comes from a noisy accelerometer signal and must be filtered; the reconstructed thrust inherits the rotor-speed sensing delay. The estimate therefore exists only after the disturbance has begun to act. No amount of tuning removes this: it is the structural property that motivates prediction. Filtering costs time in a predictable way: a second-order Butterworth low-pass filter with cut-off f_c delays slowly varying signals by $\sqrt{2}/(2\pi f_c)$, about 7.5 ms at $f_c = 30\,Hz$ (a textbook value, given for scale). The INDI literature adds a second requirement: the measured acceleration and the thrust it

is compared with must pass *identical* filters, so that both carry the same lag and the difference is not corrupted by a phase mismatch [17, 15]. Here both pass the same second-order Butterworth filter at 30 Hz, designed for and run at the controller's 500 Hz, so the delay figure above applies to both.

It absorbs model error. Any error in m or k_t appears in a_{model} and is attributed to f_{res}. A miscalibrated thrust constant produces a residual that looks like a constant interaction force, as the first example above shows. An accelerometer bias enters a_{meas} in the same way; Smeur et al. note that an INDI position loop is sensitive to such a bias, because a measurement offset becomes an offset in the achieved acceleration [15]. This is why the airframe constants are identified once and frozen, and why the null scenarios matter: a residual that persists when no neighbour is present is model error, not interaction.

It is a measurement of the force, not of the controller's success. Because a_{model} is formed from the thrust the rotors actually produce, as reconstructed from their measured speeds, a controller that compensates the disturbance changes f_i and therefore a_{model}, but not the difference (3.3) at a given state. The same property is stated by Shi et al. for their identical definition: compensating a learned force "will only affect" the commanded thrust "and not the observed" interaction force [14]. What compensation does change is the state the vehicle visits, and with it the residual that state produces.

3.3.2 Sign convention in the control law

The compensation enters with a sign that is easy to get wrong and whose error is not obvious in flight. From (3.1), holding a desired acceleration a_{des} (the acceleration the trajectory demands, without gravity and without the residual) requires

$$f_i R_i e_3 \;=\; m\, a_{\text{des}} + mge_3 - f_{\text{res},i}, \tag{3.4}$$

obtained by setting $\dot{v}_i = a_{\text{des}}$ in (3.1) and solving for the thrust vector. The desired-acceleration vector passed to the thrust-and-attitude allocation therefore carries $-f_{\text{res},i}/m$. The same sign appears, written in each paper's own frame, in the controllers that compensate a learned or measured force: Neural-Swarm and Neural-Swarm2 subtract the predicted force from the desired force [14, 13], Tal and Karaman's linear-acceleration INDI increment is algebraically the commanded acceleration, minus gravity, minus the estimated external acceleration [17], and the geometric controller of Cobo-Briesewitz et al. and the flatness-based controller of Hsieh et al. subtract the residual before forming the thrust [3, 7]. The residual is *subtracted*, whether it was measured (Section 3.3) or predicted (Section 3.4), and the flown controller does so by construction: it has no configurable sign (Chapter 4).

Adding it instead does not merely fail to compensate: it doubles the disturbance, because the controller then commands a thrust error equal and opposite to the correction it should have made. The resulting behaviour is a stable, well-controlled vehicle that tracks slightly worse in formation. An uncompensated vehicle can look the same. In single-vehicle flight the error is hard to see, since without a neighbour the residual consists only of the vehicle's own small model error, whose effect is doubled as well but remains small. With the sign correct, position-INDI compensation must *reduce* the formation-induced displacement. With the sign inverted, the displacement would be larger, approximately twice the uncompensated value in a quasi-static stack in the linear regime. With thrust saturation or filter delay the factor need not be exactly two.

The factor of two follows from one line of arithmetic. Take the vertical axis alone, a proportional-derivative position loop with stiffness k_p and no integral action, and a constant residual acceleration d pressing the vehicle down. At rest the loop balances the residual with a position error: $k_p e = d$ without compensation, so $e = d/k_p$; $k_p e = d - d = 0$ with the measured residual subtracted; and $k_p e = d + d$ with it added, so $e = 2d/k_p$. For example, with $d = 1\,\mathrm{m/s^2}$ and $k_p = 30\,/\mathrm{s}^2$ (illustrative values) the sag is 3.3 cm, zero, or 6.7 cm. The argument assumes no integral action on that axis: an integrator drives the steady-state error to zero whatever the sign, and the difference between the signs then appears only in the transient. The flown controller has such an integrator on the vertical axis in every strategy (Chapter 4), so for it the argument describes the transient, not the steady state.

3.4 Predicting the residual from relative state

The interaction force on vehicle i is a function of the configuration of the team relative to it, and, near the ground, of the vehicle's own height and velocity,

$$f_{\mathrm{res},i} \;\approx\; \mathcal{F}(\{\, p_j - p_i,\; v_j - v_i \,\}_{j \neq i},\; z_i,\; v_i)\,, \tag{3.5}$$

where $z_i = e_3^\top p_i$ is the height above the floor. Equation (3.5) is what a predictive strategy learns. Two structural properties are required of any parameterisation of $\mathcal{F}$, and both follow from the physics rather than from convenience. The value must not depend on the order in which neighbours are enumerated, and it must be defined for a varying number of them, since a vehicle in a three-robot team has two neighbours and one in a pair has one.

The neighbours' positions reach each vehicle over the radio, with time stamps, as part of the motion-capture broadcast that the flight stack already uses; the onboard model therefore needs no additional communication. That stream carries positions only, so each vehicle estimates a neighbour's velocity by finite differences of successive positions: the first sample of a neighbour gives zero velocity, a repeated time stamp reuses the last estimate, and a gap of 500 ms or more restarts the estimate. At most three neighbours are considered.

3.4.1 Deep-sets parameterisation

The parameterisation used here satisfies both properties by construction:

$$\hat{a}_{\mathrm{res},i} \;=\; e_3\, \mathrm{sat}_{\bar{a}}\left(\frac{c}{m}\, \rho\left(\phi_G(-z_i,\, -v_i) + \sum_{j \in \mathcal{N}_i} \phi_S(p_j - p_i,\; v_j - v_i) \right) \right), \tag{3.6}$$

where ϕ_S is applied to each admitted neighbour independently, ϕ_G encodes the vehicle's own height and velocity, and ρ maps the summed embedding to a scalar. The set $\mathcal{N}_i$ contains the neighbours that pass the input gate of Section 3.4.3. The scalar is the vertical component of the interaction force in grams-force, the unit in which Neural-Swarm2 reports the learned force [13]; the factor $c = g/1000$ converts it to newtons and the division by m to an acceleration, and $\mathrm{sat}_{\bar{a}}$ limits its magnitude to $\bar{a}$ (Section 3.4.3). The horizontal components are zero by construction. Summation is permutation-invariant and accepts any number of terms, so one parameter set serves teams of different sizes. The output is an acceleration rather than a force so that it is directly comparable with the vertical component of the measured a_{res} of (3.3); the two quantities are in the same units, in the same frame, at the same point in the control loop.

A toy example. The functions in this paragraph are invented for illustration and are not the trained networks. Let a scalar embedding $\phi(\Delta z) = 1/\Delta z^2$ describe a neighbour directly above at height difference Δz (in metres, units dropped), and let $\rho(s) = s/(1+s/10)$. Two neighbours at $\Delta z = 0.3$ and 0.5 give $\phi = 11.1$ and 4.0, whose sum 15.1 is the same in either order; this is the permutation invariance. The aggregate is $\rho(15.1) = 6.0$, whereas adding the two single-neighbour effects would give $\rho(11.1) + \rho(4.0) = 5.3 + 2.9 = 8.2$. Because ρ is nonlinear, the summed embedding can express an aggregate that is not the sum of the pairwise effects; it cannot express an aggregate that depends on how the two neighbours are arranged relative to *each other* beyond what their separate embeddings encode.

What is borrowed and what is specific to this thesis. The *family* is the permutation-invariant deep set used by Neural-Swarm and Neural-Swarm2 [14, 13], and the architecture is Neural-Swarm2's own: Neural-Swarm2 uses one ϕ per neighbour type, one ρ per own type and treats the environment as a further neighbour, to capture ground effect [13]. The weight layout keeps Neural-Swarm2's five weight sets, of which only the small-vehicle pair and the ground branch are evaluated, because every vehicle in this study is a Crazyflie. The layers are: ϕ_S maps $6 \to 25 \to 40 \to 40 \to 20$ (relative position and velocity of one neighbour), ϕ_G maps $4 \to 25 \to 40 \to 40 \to 20$, and ρ maps $20 \to 40 \to 40 \to 40 \to 1$, with rectified-linear activations in the hidden layers and a linear output, because an interaction force may point either way. These are the widths Neural-Swarm2 reports, with a hidden dimension of 20 [13]. The stored weight array has 19297 entries, of which 11461 lie on the evaluated path; the weights are fixed in the flight software. What is specific to this thesis is the training: the network is trained on dedicated two-vehicle training flights under Strategy 0 (Chapter 4), with a robust (Huber) loss on the vertical component of the measured residual, and spectral normalisation, which Neural-Swarm and Neural-Swarm2 apply [14, 13], is not used. This is not the network of Cobo-Briesewitz et al. [3]. An architecture ablation would be a separate study and is out of scope unless time remains after the comparison.

The network is not evaluated at every control step. It runs on every tenth tick of the 1 kHz main loop, that is at 100 Hz or every fifth step of the 500 Hz controller, and its output is held in between; the prediction is computed and recorded whether or not it is used by the controller (Chapter 4). The sizing is discussed in Chapter 4; what matters here is that (3.6) is the model, and the layer widths are an implementation choice within it.

These equations are the flown ones. Equation (3.6), the layer widths above, the input gate of Section 3.4.3 and the output bound are those of the flight software, not an idealisation of them. The same applies to (3.3) and the sign convention of Section 3.3.2. If the flight software changes, this chapter changes with it; a modelling chapter that has drifted from the flight software describes a system that was never flown.

3.4.2 The superposition assumption

In words: each neighbour is turned into a short list of numbers by the network ϕ_S, the lists of all admitted neighbours are added to the list for the ground, and a second network ρ turns the sum into one force. The short list is called the latent space (or embedding). Equation (3.6) assumes the aggregate interaction is a sum of per-neighbour contributions in the latent space of ϕ_S. This is more general than summing forces directly, ρ is free to be nonlinear in the sum, but it is still an assumption, and the literature states plainly

that it does not hold exactly. Shi et al. [14] report that with more than two vehicles the aerodynamic effect is not a simple superposition of each pair, and Gielis et al. [4] motivate a learned aggregation model, measured on a load stand, precisely on the grounds that a single-vehicle downwash model is unlikely to suffice for predicting aggregate forces. Two further observations bear directly on the transfer question of this thesis. Shi et al. found that a network trained on two-vehicle data alone flew worse than the uncompensated baseline on three- and four-vehicle swaps, "since the training data does not include data on superpositions" [14]. And Hsieh et al. nevertheless model the aggregate on three vehicles as a pairwise summation of a model trained on two, citing the same load-stand measurements [6]; the linear regime that Gielis et al. identify is consistent with that choice, and their nonlinear regime is not [4].

The assumption is retained here for two reasons. It is what makes one trained model applicable to teams of two and three without retraining, which the comparison requires. And a controller does not need an exact model: it needs one whose residual after compensation is smaller than the residual before it. The relevant question is therefore not whether (3.6) is exact. It is not. The question is how large the resulting error is on a deployed model. The multi-robot results chapter measures that.

3.4.3 Input conditioning

One gate is applied to the inputs of ϕ_S, and it is part of the model rather than defensive programming, because it changes the function being learned. A neighbour j enters the sum in (3.6) only if

$$|x_j - x_i| < 0.2\,m, \qquad |y_j - y_i| < 0.2\,m, \qquad |v_{x,j} - v_{x,i}| < 1.5\,m/s.$$

Neural-Swarm2 defines a vehicle's neighbours through an interaction volume that it chooses as "a cuboid based on observed interactions" [13]; the numerical bounds above are those of the authors' public code, including the asymmetry that the relative velocity is gated along x only. Three consequences follow, and each matters for the experiments. A neighbour that is offset laterally by more than 0.2 m contributes nothing, at any vertical separation: the model treats the wake as a column about 0.4 m wide, which is consistent in kind with the load-stand observation that a multirotor's downwash stays concentrated within about a body radius over a long vertical distance [4]. There is no vertical cut-off: a neighbour well above is admitted, and its effect is left to the learned ϕ_S. And there is no near-field clamp: the model is evaluated at whatever separation occurs, so it is the scenario library and its safety limits (Chapter 5) that keep the inputs within the range the training data covered. A learned function is least trustworthy exactly where the physics is strongest, and that range limit is what bounds the extrapolation.

The output is bounded as well: the magnitude of the vertical prediction is limited to $\bar{a} = 8\,\mathrm{m/s^2}$, comfortably above the interaction accelerations expected between vehicles of this size, and far below what an ill-conditioned network can emit; a non-finite output is replaced by zero and flagged in the same way. This is not a hypothetical failure mode. In simulation, Gu et al. [5] report a radial-basis-function estimator crashing the vehicle under periodic wind, and an extreme learning machine becoming unstable on landing under ground effect; their echo state network and Gaussian process did not. The authors say the failure *may be attributed* to the random initialisation of the input-to-hidden weights, which *"can produce excessively large prediction values"*. Their vehicle is a single quadrotor and their disturbances are wind and ground effect, so this is not evidence about inter-vehicle interaction, but it is evidence that an unbounded learned term can destabilise a controller.

A saturating output converts that failure into a bounded and, because saturation is logged, a visible one.

The training data must pass through the same gate as the deployed model, and the training pipeline applies it. A model fitted to ungated inputs is fitted to a different distribution than the one it will meet, and the discrepancy is invisible until predicted and measured residuals are compared in flight.

3.4.4 Normalisation

Input scaling is folded into the first layers of the network at export rather than applied onboard. There are two such layers, the first layer of ϕ_S and the first layer of ϕ_G, and each receives its own statistics, because a neighbour's relative state and a vehicle's own height and velocity follow different distributions. For weights W_1 and bias b_1 trained on inputs normalised by mean μ and standard deviation σ,

$$W_1 \frac{x-\mu}{\sigma} + b_1 \;=\; \underbrace{(W_1 \oslash \sigma)}_{W_1'} x + \underbrace{(b_1 - (W_1 \oslash \sigma)\,\mu)}_{b_1'}, \tag{3.7}$$

with $\oslash$ denoting column-wise division. The two forms are arithmetically identical, so the deployed network computes the same function as the trained one while the flight software carries no normalisation constants that could fall out of step with the model. For example, a first-layer weight of 2 applied to a height normalised with $\mu = 1\,m$ and $\sigma = 0.5\,m$ becomes the weight 4 applied to the raw height, with -4 added to the bias; the neuron receives the same input in both forms.

3.5 Assumptions and where they fail

Table 3.1 collects the assumptions this formulation makes. Each is stated with the condition under which it is expected to break, so that an experimental result can be attributed to a specific assumption rather than to the model as a whole.

Two entries deserve emphasis because they are not merely caveats but hypotheses the experiments test. The superposition assumption is known to be imperfect, and the experimental question is the size of the penalty rather than its existence. And the training-distribution assumption is exactly what separates the predictive family from the reactive one: a measurement has no training distribution to leave. Cobo-Briesewitz et al. [3] observe precisely this effect for their purely learned method in the no-payload case, attributing a drop in performance on the circular trajectory to its being out of distribution relative to the training data.

3.6 Summary

The residual of (3.1)–(3.2) is what this study measures and records, with one definition for every strategy: Strategies 1 and 3 use the measurement, Strategies 2 and 3 a prediction of its vertical component, and Strategy 0 neither. The residual can be recovered online from (3.3), which needs no model of the interaction but requires rotor-speed sensing and is available only after the disturbance acts; or its vertical component can be predicted from (3.6), which acts ahead of the disturbance but is valid only within its training distribution, sees only neighbours inside its input gate, and assumes an aggregation that is known to be approximate. Both are expressed as accelerations in the world frame, and both enter the control law at the same point with the same sign, (3.4).

Table 3.1: Assumptions of the residual formulation, the conditions under which each is expected to fail, and the consequence for the measured residual a_{res} and the prediction $\hat{a}_{\text{res}}$.

Assumption	Where it fails	Consequence
Rigid body; no aeroelastic deformation	Not expected to fail at this scale	—
m, J, k_t known and constant	Rotor wear; temperature; miscalibration	Model error appears as a constant residual; bounded by the null scenarios (second phase), provisionally by the repeat-to-repeat variation
Accelerometer unbiased	Sensor bias or drift	Appears as a constant residual; bounded by the null scenarios (second phase), provisionally by the repeat-to-repeat variation
Residual is dominated by inter-vehicle interaction	Near the ground; near walls	Flight volume is bounded away from both (Chapter 5)
Neighbour relative states are known accurately	Motion-capture dropout; latency; velocity differenced from broadcast positions	Prediction degrades; measurement is unaffected, which is itself a comparison result
The interaction is negligible outside the input gate	Neighbours offset laterally by more than 0.2 m that still disturb the vehicle	Prediction is zero where a residual is measured; visible as a prediction–measurement gap
Aggregate interaction superposes in latent space	Three or more vehicles, overlapping wakes	**Known to be imperfect** [14, 4]; magnitude measured in the second, three-vehicle phase (multi-robot results chapter)
Interaction lies within the training distribution	Unseen separations, faster relative motion	Prediction degrades while measurement does not; this is the crossover of **RQ2**
Rotor-speed measurement available	Rotor-speed sensor absent, or a rotor reading missing	Residual terms read zero or are wrong — silent, and checked every session

That common definition, together with the common controller of Chapter 4, is what makes the comparison in the two-robot results chapter a comparison rather than a collection of separately-tuned controllers. Chapter 4 describes the strategies built on it, and the point at which each injects its knowledge of the interaction.

Chapter 4

Control Architectures

Chapter 3 defined the residual wrench and the two ways of obtaining it, by measurement and by prediction. This chapter describes the control strategies compared in this thesis and, for each, the point at which knowledge of the interaction enters the control law.

Five strategies are defined (Table 4.1). Four of them, Strategies 0–3, are one controller: a geometric tracking controller to which three independent switches add a measured residual force, a measured residual moment and a learned residual force. Strategy 0 has all three switches off and is the uncompensated baseline ("uncompensated" means that it does not use the residual); Strategy 1 uses the measured residual (incremental nonlinear dynamic inversion, INDI); Strategy 2 uses the learned residual; Strategy 3 uses both. Strategy 4 is a different controller, a feedback-linearising law with a learned residual, and is described as a reference design.

Because Strategies 0–3 are one controller, they share the position and attitude gains, the filters and the physical parameters, and differ only in which switches are set. What makes the comparison fair is therefore both the experimental conditions held constant and the shared control law; Section 4.7 states precisely what is shared and what the comparison still does not guarantee.

Each strategy is described in two parts, kept separate: the law that is flown, and the published method it follows, with the deviations from that method listed.

4.1 The common controller

Strategies 0–3 share one position loop and one attitude loop. The switches add terms to them; they do not replace either loop. The equations are stated once in this section.

4.1.1 Position loop

The desired acceleration is

$$a_d \;=\; a_{\mathrm{ff}} + g e_3 + K_p e_p + K_v e_v + k_{i,z}\, e_3 \int e_3^{\top} e_p \,\mathrm{d}t, \tag{4.1}$$

with $e_p = p_{\mathrm{des}} - p$, $e_v = v_{\mathrm{des}} - v$ and a_{ff} the trajectory's acceleration. The integral acts on the vertical axis only, and its state is limited. The switches then form the commanded acceleration

$$a_{\mathrm{cmd}} \;=\; a_d \underbrace{- a_{\mathrm{res},f}}_{\text{switch A}} \underbrace{- \hat{a}_{\mathrm{res}}}_{\text{switch C}}, \tag{4.2}$$

Table 4.1: The strategies. Switches A and B add the measured residual force and moment, switch C the learned residual force; NN denotes the learned residual and FBL feedback linearisation. "Knows" refers to the residual: whether the controller has a measurement of it, a prediction of it, both, or neither. "Enters at" is the quantity the residual is subtracted from, and "Rotor speed" says whether the strategy needs rotor-speed sensing at run time.

#	Strategy	Knows	Switches	Rotor speed	Implemented
0	Geometric baseline	nothing	—	no	yes
1	INDI	measured	A, B	yes	yes
2	Geometric + NN	predicted	C	no[a]	yes
3	INDI + NN	measured, predicted	A, B, C	yes	yes
4	FBL + NN	predicted	—	no[a]	no

[a] Not at run time. Producing the training labels requires the measured residual (3.3), and therefore rotor-speed sensing on the vehicles that collect the data, as for the learned method of Cobo-Briesewitz et al. [3]. The evaluation of Chapter 5 records rotor speeds on every flight.

where $a_{\mathrm{res},f}$ is the measured residual (3.3) after the filters of Section 4.3 and $\hat{a}_{\mathrm{res}}$ the learned prediction of Section 4.4. Both residual terms are subtracted, the sign derived in Section 3.3.2. The part of (4.1) without ge_3 is the a_{des} of Chapter 3.

The commanded thrust vector is $m\,a_{\mathrm{cmd}}$. Its projection on the body z-axis is the collective thrust, $f = m\,a_{\mathrm{cmd}}^{\top} R e_3$, and its direction, together with the desired yaw ψ_{des}, fixes the desired attitude: with $y_c = [-\sin\psi_{\mathrm{des}}, \cos\psi_{\mathrm{des}}, 0]^{\top}$, the columns of R_{des} are

$$x_{\mathrm{des}} = \frac{y_c \times a_{\mathrm{cmd}}}{\|y_c \times a_{\mathrm{cmd}}\|}, \qquad y_{\mathrm{des}} = \frac{a_{\mathrm{cmd}} \times x_{\mathrm{des}}}{\|a_{\mathrm{cmd}} \times x_{\mathrm{des}}\|}, \qquad z_{\mathrm{des}} = x_{\mathrm{des}} \times y_{\mathrm{des}},$$

so that the body z-axis points along a_{cmd} and the desired heading fixes the rotation about it. Because the residual terms are inside a_{cmd}, they change both the thrust and the desired attitude.

4.1.2 Attitude loop

The attitude error and the rate error are those of Lee et al., $e_R = \frac{1}{2}\left(R_{\mathrm{des}}^{\top} R - R^{\top} R_{\mathrm{des}}\right)^{\vee}$ and $e_\omega = \omega - \omega_r$ with $\omega_r = R^{\top} R_{\mathrm{des}}\,\omega_{\mathrm{des}}$, where the desired angular velocity ω_{des} and angular acceleration $\dot{\omega}_{\mathrm{des}}$ follow from the trajectory's jerk and snap by differential flatness. The torque is

$$\tau_{\mathrm{geo}} = -K_R e_R - K_\omega e_\omega - K_I \int e_R \,\mathrm{d}t + \omega \times J\omega - J\left(\hat{\omega}\,\omega_r - R^{\top} R_{\mathrm{des}}\,\dot{\omega}_{\mathrm{des}}\right), \tag{4.3}$$

and switch B adds to it the measured-moment correction of Section 4.3. Here K_R and K_ω are diagonal gains from attitude and rate error to torque, in Nm per unit of e_R and Nms per unit of e_ω, and K_I is the gain of an integral of the attitude error. Apart from that integral, (4.3) is the torque law of the source with its feedforward bracket, given in Section 4.2.

4.1.3 Parameters

Table 4.2 lists the parameters of the common controller. All of them are the same in Strategies 0–3.

Table 4.2: Parameters of the common controller, identical in Strategies 0–3.

Quantity	Symbol	Value
Position gain, all axes	K_p	$7\,/\mathrm{s}^2$
Velocity gain, all axes	K_v	$4\,/\mathrm{s}$
Vertical integral gain	$k_{i,z}$	$1.5\,\mathrm{s}^{-3}$
Attitude gain	K_R	$\mathrm{diag}(0.007,\ 0.007,\ 0.008)\,\mathrm{Nm}$
Rate gain	K_ω	$\mathrm{diag}(0.00115,\ 0.00115,\ 0.002)\,\mathrm{Nms}$
Attitude integral gain	K_I	0.03 on each axis
Mass	m	42.7 g
Inertia	J	$\mathrm{diag}(16.57,\ 16.66,\ 29.26) \times 10^{-6}\,\mathrm{kgm}^2$
Rotor thrust constant	k_t	$3.912 \times 10^{-8}\,\mathrm{Ns}^2$ per rotor
Arm length (centre to rotor)	l	0.050 m
Rotor drag torque to thrust	κ	0.00569 m
Residual filters	—	second-order Butterworth, 30 Hz

4.1.4 Rates

The controller executes at 500 Hz, on every second tick of the 1 kHz main loop of the flight software. The motion-capture pose reaches the vehicles at 100 Hz, and the learned residual is evaluated on every tenth main-loop tick, that is 100 Hz, and held in between.

4.1.5 Runtime selection

The switches are parameters set at run time, not compile-time options: switch A adds the measured residual force in (4.2), switch B the measured-moment correction to (4.3), and switch C the learned residual in (4.2). Strategy 0 has none set, Strategy 1 has A and B, Strategy 2 has C, and Strategy 3 has A, B and C. Each strategy is one frozen set of parameter values, sent to the vehicle before take-off and read back from it. The learned prediction is computed and recorded whatever switch C says; the switch decides only whether it enters (4.2). Switches A and B need the rotor-speed source and are inert without it.

The two vehicles of a formation run the same controller. The upper vehicle always flies Strategy 0; the strategy under comparison, including any learned term, is flown by the lower vehicle, the one that meets the downwash.

Setting A or B alone gives two further configurations. They are diagnostic, used in the two-robot results chapter to attribute an effect to the force or the moment term, and are not strategies.

How take-off and landing are flown for the strategies that use switches A and B is fixed at the hardware gate (Section 5.5.1); where it differs from the flight phase compared, the analysis windows of Section 5.5.4 exclude it.

Runtime selection matters more than it may appear. Comparing strategies that require different firmware introduces a confound that cannot be ruled out afterwards: a flash cycle changes more than the control law, and strategies flown on different days meet different battery states, temperatures and motion-capture calibrations. With one controller and run-time switches the strategies can be interleaved within a session, which makes the comparison a paired one.

4.2 Strategy 0: uncompensated geometric baseline

The law. Equations (4.1)–(4.3) with all three switches off. The controller has no knowledge of the interaction.

Reference. Geometric tracking on $SE(3)$ [10], which tracks a position trajectory together with a single heading direction, four outputs, not a full six-degree-of-freedom pose. The attitude error of (4.3) is that work's $e_R = \frac{1}{2}(R_d^\top R - R^\top R_d)^\vee$. Lee et al. write the translational dynamics with the third axis pointing *down*, $m\dot{v} = mge_3 - fRe_3$, and define the position error as $e_x = x - x_d$, the opposite of the $e_p = p_{\text{des}} - p$ used here [9]. Position-loop terms therefore change sign when transcribed into this thesis' east-north-up convention, whereas the attitude error and the torque law, written in body coordinates, carry over unchanged. The torque law of the source is

$$M = -k_R e_R - k_\Omega e_\Omega + \Omega \times J\Omega - J\Big(\hat{\Omega}\, R^\top R_d \Omega_d - R^\top R_d \dot{\Omega}_d\Big), \tag{4.4}$$

with $e_\Omega = \Omega - R^\top R_d \Omega_d$; the symbols map to those used here as $k_R \to K_R$, $k_\Omega \to K_\omega$, $\Omega \to \omega$, $R_d \to R_{\text{des}}$ and $\Omega_d \to \omega_{\text{des}}$.

Deviations. Equation (4.3) contains every term of (4.4) and adds an integral of the attitude error; the position loop adds a vertical integral; the desired attitude is built from the direction of a_{cmd} and the desired heading as above, rather than by the heading projection of the source; and ω_{des} and $\dot{\omega}_{\text{des}}$ are computed from the trajectory's jerk and snap. The integral terms mean that no stability result of the source transfers to the flown law unexamined. The inertia used by the controller is the one identified for the earlier Crazyflie 2.0 airframe, not one identified for the vehicles flown here; an inertia error affects the feedforward and gyroscopic terms of (4.3) and, with switch B, the moment correction.

What the source proves. Not global asymptotic tracking. With the attitude error function Ψ, exponential stability is established on $\Psi(0) < 1$, initial attitude errors below $90\,^\circ$, and almost-global exponential attractiveness on $1 \le \Psi < 2$, below $180\,^\circ$. This thesis claims neither for the flown law: the baseline is flown as a reference condition, not as a certified controller. Here $\Psi(R, R_d) = \frac{1}{2}\operatorname{tr}(I - R_d^\top R)$, which equals $1 - \cos\theta$ for a rotation by the angle θ between R and R_d, so $\Psi = 1$ at $90\,^\circ$ and $\Psi = 2$ at $180\,^\circ$ [9].

Role. Strategy 0 is the uncompensated baseline of Table 4.1. It is flown in every scenario so that "how much did compensation help" has a denominator, and so that the interaction can be separated from the model error discussed in Section 3.3.1. It is also the controller of the dedicated training flights from which the learned residual is obtained (Section 4.4); those flights are separate from the comparison flights, so that the learned model is never evaluated on its own training data.

4.3 Strategy 1: INDI

The law. Switches A and B. Switch A subtracts the measured residual force in (4.2). It is formed as in (3.3) from two accelerations: the acceleration the rotors are producing, computed from the four measured rotor speeds,

$$a_{\text{rot}} = \frac{1}{m}\Big(\sum_{k=1}^{4} k_t \varpi_k^2\Big) Re_3 - ge_3,$$

and the measured acceleration a_{meas}, the accelerometer reading rotated into the world frame with the estimated attitude and with gravity removed. Both pass identical second-order Butterworth low-pass filters at 30 Hz, sampled at 500 Hz, and $a_{\text{res},f} = a_{\text{meas},f} - a_{\text{rot},f}$. Switch B adds to the torque (4.3) the measured-moment correction

$$\tau = \tau_{\text{geo}} + (\tau_{\text{cur},f} - J\alpha_f), \tag{4.5}$$

where τ_{cur} is the torque the rotors are producing, computed from the four rotor thrusts $k_t\varpi_k^2$, the arm length and the rotor drag-to-thrust ratio, and α is the angular acceleration obtained by differencing the gyro; both pass the same 30 Hz filter. The correction is added to the full geometric torque; it does not replace the torque law.

Why the terms reject the disturbance. For the force, substituting $a_{\text{res},f}$ into $fRe_3 = m\,a_{\text{cmd}}$ gives

$$fRe_3 = m\,(a_{\text{des}} + ge_3) - m\,(a_{\text{meas},f} - a_{\text{rot},f}) = f_{\text{cur},f}Re_3 + m\,(a_{\text{des}} - a_{\text{meas},f}),$$

with $f_{\text{cur}} = \sum_k k_t\varpi_k^2$ the thrust being produced: the new thrust vector is the current one corrected by the acceleration error, the linear-acceleration increment of Tal and Karaman and of Smeur et al. [17, 15]. For the moment, defining the reference angular acceleration that the geometric law commands,

$$J\alpha_{\text{ref}} = \tau_{\text{geo}}, \tag{4.6}$$

turns (4.5) into

$$\tau = \tau_{\text{cur},f} + J\,(\alpha_{\text{ref}} - \alpha_f)\,, \tag{4.7}$$

the angular-acceleration increment of Tal and Karaman [17]: the controller compares the angular acceleration it wants with the one it gets, and changes the torque currently applied by the difference. Both increments correct relative to what is actually being achieved, so they reject a disturbance that appears in the measured accelerations without a model of it. Tal and Karaman state the assumptions this needs: the disturbance changes slowly compared with the filters and the sample period, and the measured quantities are filtered alike [17]. The filters are the source of the delay identified in Section 3.3.1.

Reference. The structure is that of the INDI controller evaluated by Cobo-Briesewitz et al.: the geometric law of Lee et al. with the residual force and moment, estimated online from the inertial measurements and the rotor speeds, subtracted as feedforward terms, their eqs. (8a), (8b), (10a) and (10b) [3]. The incremental principle and the requirement of identical filters follow Tal and Karaman [17] and, for the cascade of a linear-acceleration loop over the attitude loop, Smeur et al. [15].

Deviations. From Cobo-Briesewitz et al.: the moment estimate omits the gyroscopic term of their eq. (10b), so the correction is $\tau_{\text{cur},f} - J\alpha_f$ rather than the full residual moment, and the gyroscopic coupling, which (4.3) already compensates, enters the correction a second time; the vertical position integral and the attitude integral of the common controller have no counterpart in their law; there is no payload term; and the rotor speeds come from an optical rotor-speed sensor with one thrust constant for all rotors. From Tal and Karaman: the rotor speeds are not measured by encoders on the motors [17], and Tal and Karaman note that the incremental structure removes the need for integral action in their design [17], whereas here the integrals of the common controller remain. No claim is made that the flown law reproduces the tracking performance reported in either work.

Requirements. The rotor-speed source, without which switches A and B are inert; the method needs no training data.

4.4 Strategy 2: geometric with a learned residual

The law. Switch C. The deep-sets prediction $\hat{a}_{\text{res}}$ of (3.6) is subtracted in (4.2). The network is evaluated on the vehicle at 100 Hz from the relative states of the neighbours: their positions arrive over the radio with time stamps, and their velocities are obtained by finite differences (Section 3.4). The prediction is a vertical acceleration; the horizontal residual is not compensated. The weights are trained offline on the dedicated training flights of Strategy 0 and are fixed in the flight software for the whole comparison. Its execution time on the vehicle is measured at the hardware gate.

The prediction is computed and recorded *whether switch C is set or not.* On a flight in which it does not drive the controller, predicted and measured residuals are two independent quantities, and their comparison measures the model's accuracy. Once the prediction drives the controller, it has itself moved the vehicle, and the comparison no longer isolates model quality. Both kinds of flight are therefore needed, and Chapter 5 schedules them.

Training data. The model is trained on dedicated flights under Strategy 0, never on the comparison flights: a grid of vertical separations, lateral offsets and speeds in the scenarios of the library, and randomised waypoint flights planned offline, in which the two vehicles are planned together, the upper one stays at least 0.25 m above the lower one, and the goals of the upper vehicle are biased towards the lower vehicle's horizontal position so that the pair spends most of the time inside the network's input gate. Every such flight passes the same safety check as the scenarios of the library. The labels are the vertical component of the measured residual (3.3), computed after the flight from the recorded channels with the same filters as switch A, since under Strategy 0 the controller does not compute it.

Reference. Predicting the interaction from relative states with a permutation-invariant network and applying it as feedforward follows the Neural-Swarm line [14, 13]. The closest published instance of the structure used here, a learned residual added as feedforward to a geometric $SE(3)$ controller, is a secondary configuration in the hardware experiments of Hsieh et al., whose main controller is the feedback-linearising one of Strategy 4 [7].

Deviations. The network architecture is Neural-Swarm2's (Section 3.4), but four things differ from that paper. First, Neural-Swarm2 applies spectral normalisation, which its abstract associates with stability and generalisation guarantees and on which the bounded-error assumption of its analysis rests [13]; the model here is trained without it, and those guarantees are not claimed. Second, Neural-Swarm2 uses the learned residual in motion planning as well as control [13], whereas here the trajectories are planned in advance and the residual is used for control only. Third, Neural-Swarm2's tracking controller has its own attitude law and an explicit compensation of motor delay [13]; here the prediction enters the common controller unchanged. Fourth, the model is trained on this project's two-vehicle data, it predicts only the vertical component, its output is bounded at $8\,\text{m/s}^2$, at most three neighbours are considered, and it is evaluated at 100 Hz. This strategy represents the predictive family and should be read as such, not as an evaluation of Neural-Swarm2.

Requirements. Neighbour relative states and training data. No rotor-speed sensing at run time; the training labels require it on the vehicles that collect the data.

4.5 Strategy 3: INDI with a learned residual

The law. Switches A, B and C together: the measured residual force and moment of Strategy 1 and the learned residual of Strategy 2, on the common controller. In the position loop, (4.2) subtracts both the filtered measurement $a_{\mathrm{res},f}$ and the prediction $\hat{a}_{\mathrm{res}}$, and the two are *added*, not partitioned; in the attitude loop, (4.5) applies. This is the definition of Strategy 3 in this thesis.

Two estimates of one disturbance. The measured and the predicted term act on the same vertical disturbance, the residual force of the interaction, and the question is how they combine. The measured residual is formed from the rotor speeds the vehicle actually has (Section 3.3), so it equals the residual force at the current state whatever the controller commands: a prediction that raises the thrust raises a_{rot} and a_{meas} alike and leaves a_{res} unchanged. The two terms therefore add. The increment form shows what that means. With switch C, the force increment of Section 4.3 becomes

$$f Re_3 = f_{\mathrm{cur},f} Re_3 + m\left(a_{\mathrm{des}} - \hat{a}_{\mathrm{res}} - a_{\mathrm{meas},f}\right), \tag{4.8}$$

and when the thrust has settled, $f = f_{\mathrm{cur}}$, the filtered acceleration satisfies $a_{\mathrm{meas},f} = a_{\mathrm{des}} - \hat{a}_{\mathrm{res}}$. For the INDI loop the prediction is therefore not a second estimate of the disturbance, which the measurement has already removed, but an offset of its acceleration reference by $-\hat{a}_{\mathrm{res}}$. In a quasi-static stack the position loop answers that offset with a position error, and the vertical integral removes it at steady state. Where the two estimates differ in time — the measurement is delayed by its filters, the prediction acts on the relative state as soon as it changes — the prediction leads the measurement, and that is where Strategy 3 can differ from Strategy 1. These are modelling statements derived from (3.3) and (4.2), not results.

Reference. Cobo-Briesewitz et al. propose combining a learned residual with INDI's measured one on a single vehicle, writing the residual as the sum of a learned part and a part the incremental term handles, so that the incremental term reasons about the remaining mismatch [3]. Strategy 3 is a hybrid of the same two information sources on a team, where the learned part is predicted from the neighbours' relative state.

Deviations. The learned part is the deep-sets model of Strategy 2, predicted from the neighbours' relative states and limited to the vertical component, not a model of the vehicle's own dynamics; and the measured residual is not reduced by the prediction before it is used, so the two terms add, with the consequence derived above. The deviations of Strategies 1 and 2 apply as well.

Requirements. Those of Strategies 1 and 2 together: the rotor-speed source, the neighbours' relative states and training data.

4.6 Strategy 4: feedback linearisation with a flatness-preserving residual

Implementation status. **Not implemented** in the flight software. The strategy is described as a reference design. Its absence is a scope limitation, not a failure of the comparison (the discussion chapter).

Reference. Hsieh et al. [7], building on the theory of Yang et al. [18]. The distinguishing property is where the prediction enters: not as a feedforward term added to a desired acceleration, but inside a feedback-linearising law whose validity depends on the augmented system remaining differentially flat. A generic learned residual can destroy that flatness; residuals with a lower-triangular structure, a structured correction inside the state derivative, not a free wrench, preserve it, and with it the original flat output [18]. That theory is established in simulation on a planar quadrotor; the hardware demonstration is Hsieh et al.'s. In Hsieh et al. the structure takes a simple form: the learned residual may depend only on the positions and velocities of the vehicles, not on thrust or attitude, so that thrust and attitude can still be computed algebraically from the flat outputs, position and yaw, and their derivatives [7].

Its relevance to this comparison is that it is the strongest published counter-argument to treating "predictive compensation" as one method. Hsieh et al.'s abstract reports a 31 % reduction in average tracking error. On hardware the body reports 24 % RMSE for ROM-FBL over nominal FBL and a further 29 % for learned FBL over ROM-FBL, and an average RMSE of about 5 cm that the authors describe as matching the reported performance of the nonlinear model predictive controller of Chee et al.; the direct comparison with that controller, using the same learned residual, is in simulation, where the flatness-based controller needs about one twentieth of the computation time [7]. The abstract adds that tight formation flight was achieved with under 30 s of training data and a 5 ms loop rate; on hardware that loop ran *off-board*, on a laptop streaming commands over the radio [7]. Those are the figures a predictive strategy in this class should be measured against. None of them was produced by a controller running on the vehicle.

Requirements. Neighbour relative states; training data. No rotor-speed sensing at run time; as for Strategy 2, the training labels require it.

4.7 What makes the comparison fair, and what it does not guarantee

What is shared. The airframe, the trajectories, the formations and separations, the estimator, the residual definition, and, because Strategies 0–3 are one controller, the position gains, the vertical integral, the attitude gains and the attitude integral, the filters, and the mass, inertia and thrust constant of Table 4.2. The vertical integral is active in all four, so that a constant height offset is removed in each of them and a difference in steady height does not arise merely because one strategy lacks the integral.

What differs by design. Only the switches. A difference in outcome between two of Strategies 0–3 is therefore attributable to the terms the switches add: the measured residual force and moment (A, B), the learned residual (C), or their combination.

What this does not guarantee. Four limitations are stated before results are presented rather than after. First, the shared gains are one set for all four strategies. A gain set suits some switch combinations better than others; a strategy may be disadvantaged by gains that were not chosen for it, and the comparison does not separate that effect from the effect of the switches. Second, the strategies follow published *families*, not papers reproduced exactly; the deviations are listed per strategy, and the results are statements about these switch combinations of one controller, not about the families in general. Third, the interaction of the measured and the predicted term in Strategy 3 is the one derived in Section 4.5: the two terms are added. The variant in which the measured term handles only the mismatch that remains after the prediction, as in the published hybrid [3], is not part of this comparison. Fourth, Strategy 4 is a different controller and not implemented; if it were added, the shared-controller argument would not cover it, and fairness towards it would rest on the experimental conditions alone.

4.8 Summary

Strategies 0–3 are one geometric controller with three switches: none (the uncompensated baseline), the measured residual force and moment (INDI), the learned residual (geometric + learned residual), and both (the hybrid). They share every gain, filter and physical parameter, and the upper vehicle of every formation flies the baseline. Strategy 4, a feedback-linearising law with a flatness-preserving learned residual, is a reference design and is not implemented. Chapter 5 describes the experimental design; the two- and multi-robot results chapters report what is flown.

Chapter 5

Experimental Setup and Protocol

Chapter 4 described the strategies: one controller with three switches for Strategies 0–3, and Strategy 4 as a reference design. This chapter describes the platform, scenarios, measurement chain and protocol. The study has two phases: a two-vehicle campaign first, then the three-vehicle scenarios with the same controllers (Section 5.5.3). The design removes alternative explanations for an observed difference.

Section 5.7 states the threats this design is built against, and which of them it does not eliminate.

5.1 Platform

The vehicles are Crazyflie 2.1 brushless quadrotors, nominally 41 g, with 10 cm between diagonally opposite motors. All vehicles in a team are the same platform: the gain block is shared across robots, so a mixed team would fly at least one vehicle on the wrong mass, thrust constant and inertia, which would appear in the logs as a residual and be indistinguishable from an interaction force (Section 3.3.1).

Two vehicles are needed for the first phase of the study. A third is required for the three-robot scenarios of the second phase, which cannot be run without it.

Each vehicle carries two additions that are not optional for this work:

An optical rotor-speed sensor (a rotor-speed deck), whose four rotor speeds the controller reads. Without it the applied thrust cannot be reconstructed, so the measured residual is unavailable and the residual terms *read identically zero* rather than failing loudly; Strategies 1 and 3 additionally cannot run, because switches A and B are inert. The rotor speeds that the electronic speed controllers report are recorded as well, but are not used by the controller. This is the single most consequential hardware prerequisite in the setup.

A micro SD (uSD) deck , the recording medium for the dataset, for the reasons in Section 5.4.

State estimation uses the on-board extended Kalman filter, fed with external pose from a motion-capture (mocap) system at 100 Hz. The controller executes onboard at 500 Hz within the 1 kHz main loop of the flight software; the dataset is recorded at 500 Hz, and the learned model is evaluated at 100 Hz (Chapter 4).

5.2 Flight volume

The tracked volume measures approximately 2 m × 4 m horizontally and extends to 1.7 m in height. Two distinct limits are enforced and should not be conflated: the *tracked volume*, outside which the motion-capture system cannot see the vehicles, and an *operational floor* of 0.30 m, chosen so that ground effect does not contaminate the measurement of inter-vehicle interaction.

The floor is lowered to 0.10 m for one scenario only, the near-ground pass of Section 5.3 (defined in the library, not part of the comparison campaign), whose purpose is to measure ground effect in isolation so that it can be separated from downwash rather than confounded with it.

The volume is a binding constraint rather than a formality. Several scenarios sit within approximately 10 cm of a wall at their extremes, and the scenario definitions are automatically centred and, where necessary, rotated to fit. That a configuration fits is checked before flight, not discovered during it.

The volume figures are corroborated from two independent statements but have not been tape-measured. Measure them and update this section.

5.3 The formation library

Sixteen scenarios, fixed before any data collection and not modified afterwards, are listed in Table 5.1 with their default parameters. Freezing them is what allows every strategy to meet identical excitation; a library that evolved during the campaign would make later strategies incomparable with earlier ones.

The scenarios are parameterised rather than hard-coded, separations, offsets and speeds are arguments, so that a separation sweep is the same scenario at different parameters rather than a different scenario. Each is compiled to a per-vehicle polynomial trajectory and uploaded through the standard trajectory interface before take-off. The commanded relative geometry between any two vehicles is known in advance from the scenario definition, which the analysis uses to define its windows (Section 5.5.4).

5.3.1 The null scenarios

C1–C3, the coplanar controls in which the vehicles fly side by side at equal height, are the most important scenarios in the library and the least interesting to watch. They place the vehicles in configurations where no significant inter-vehicle interaction should exist, and they exist because of the property established in Section 3.2: the residual is a catch-all, non-zero even in isolated flight, so a residual measured in formation cannot be attributed to the interaction without a reference. The physical basis for this choice is that a multirotor's downwash remains concentrated beneath it; on their load stand, Gielis et al. found the aerodynamic influence of a vehicle negligible on neighbours outside its downwash, even at small separations, while noting small, chaotic lateral forces at larger distances [4].

They serve two purposes. They bound the measurement floor, so that a reported improvement can be distinguished from noise. And they detect model error: a residual that appears when no neighbour is in the wash is a mis-identified mass or thrust constant, not an interaction, and must be corrected rather than explained afterwards. The null scenarios are three-vehicle scenarios and are flown in the second phase of the study (Section 5.5.3), so the floor they set is known only after the two-vehicle campaign. The two-vehicle results

are therefore read in two steps: provisionally against the repeat-to-repeat variation within a cell, that is, between flights of the same strategy, scenario and setting, and finally against the floor of the null scenarios. In the first phase, the flights of the hardware gate at large separation, with no neighbour in the wash, give the first check for model error (Section 5.5.1).

5.3.2 Regimes, and why the library spans them

The scenarios are not a list of poses but a deliberate spread across the interaction regimes that Research Question 2 distinguishes. A1, the two-robot vertical hover stack, and the coplanar hover controls C1 and C3 are static, and the leader–follower control C2 translates at constant speed; A2 and A3 are quasi-static; A4 and A5 sweep lateral offset; A7 sweeps separation continuously; A8 and B3 present the disturbance as a transient with a known arrival time. A model fitted on one regime and evaluated on another is what makes "outside the training distribution" a measurable condition rather than a caveat.

> **Requirement for data collection: the set must excite lateral relative motion.** The six network inputs are the relative position and the relative velocity of a neighbour, three components each. A purely vertical scenario holds the four lateral components (relative x and y, position and velocity) nearly constant, so a model trained only on such data has seen almost no variation in four of its six inputs and is extrapolating the first time a formation moves sideways. **The collection set must therefore include A4, A5 and A7 — not only the vertical stacks A1, A2 and A3.** This is not a preference about coverage; a set without them produces a model that cannot be evaluated on the scenarios it will meet.

The input gate of the network (Section 3.4.3) sharpens this requirement. A neighbour more than 0.2 m away along either horizontal axis contributes nothing to the prediction, so the coplanar controls at their default spacings (0.30 m and wider) exercise only the ground branch, A5 exercises the neighbour branch only during the part of each lap in which the two vehicles pass within that band, and A4 at its default offset of 0.10 m stays inside it. Lateral excitation matters within the band, not across the whole room.

The requirement follows from the inputs alone, and it can be checked before any flight from the commanded trajectories of the planned collection set: an input whose commanded range is near zero across the set will not vary in the data either.

5.4 Measurement and logging

5.4.1 Why the dataset is recorded onboard

Radio telemetry shares a fixed bandwidth per radio dongle among all vehicles it serves, so the rate at which each vehicle can stream its log falls as vehicles are added. Recording the dataset over radio would therefore couple the sampling rate to the team size, which is unacceptable for a study whose subject is what happens as vehicles are added.

The micro SD deck records 96 variables at 500 Hz on each vehicle independently of radio and of team size. Radio logging is retained for live monitoring and for single-vehicle tuning, where the bandwidth exists.

5.4.2 What is recorded

Position and velocity estimates and the attitude, as Euler angles and as a quaternion; gyro and accelerometer; the controller's two filtered accelerations, whose difference is the measured residual (Section 3.3); its commanded thrust and torque; its actual and desired attitude, from which the attitude error e_R is computed; its position and velocity errors; the network prediction $\hat{a}_{\text{res}}$ and its saturation flag; the commanded position, velocity and acceleration; the four motor commands; the rotor speeds as measured by the optical sensor and as reported by the motor controllers; the battery voltage; and the external pose received from motion capture.

Two of these deserve comment. The controller computes the measured residual only when switch A is set; for every other flight it is computed after the flight from the recorded accelerometer signal, attitude and rotor speeds, with the same constants and filters (Section 3.3), so one definition holds for all strategies. The network prediction is recorded *whether it is used or not*. Comparing prediction to measurement requires both on the same flight without the prediction driving the loop.

The logging configuration is fixed before data collection and shared by the collection and evaluation flights. A configuration change between the two would silently make the two halves of the dataset incomparable.

5.4.3 Time alignment across vehicles

Each vehicle timestamps its samples against its own boot, so the logs share no clock. Recording is started by a single broadcast, which aligns the vehicles to within the broadcast jitter plus one logging period. Residual error comes from propagation, task wake phase and crystal drift between vehicles; the last is a rate error and accumulates over a flight.

The merge tool therefore *measures* the alignment rather than assuming it. Each vehicle's onboard position trace is aligned against that vehicle's own commanded trajectory, which is known from the scenario definition and differs between vehicles, and the quality of the fit is reported. Where radio logs of the same flight exist, which share a common clock, cross-correlating each onboard trace against its radio trace additionally gives the offset and the drift as a cross-check. The expected alignment is a few milliseconds, set by the broadcast jitter and the logging period; the measured value is reported for every two-vehicle session.

5.5 Protocol

5.5.1 Hardware gate

No data collected before the following sequence passes is used. The gate is not an experiment; it is the check that nothing is broken, so that what is measured afterwards means something.

1. **Validate.** Single vehicle, the baseline first and then the INDI switches, on a sequence of trajectories from hover to the formation trajectories. Confirm the parameters read back as configured and fix how the INDI strategies take off and land.

2. **Freeze.** The shared gains of the common controller and the learned model's weights are frozen for the remainder of the campaign; any change before that happens here and only here, and applies to all strategies at once.

3. **Two vehicles at large separation.** Confirm that the rotor speeds and a_{res} are non-zero on both vehicles; confirm the measured time alignment; and, for the learned model, measure its execution time on the vehicle.

The order carries the reasoning. Tuning before validating means tuning against an unknown fault. Flying two vehicles before the freeze means the gains move underneath the dataset.

Stage 1 cannot confirm that the compensation terms are *correct*, only that they are not catastrophic: with no neighbour present $a_{\text{res}} \approx 0$, so a compensation term is dormant and a clean single-vehicle flight demonstrates nothing about it. Stage 3 is the first point at which the interaction residual is observable at all, which is why it is part of the gate rather than the first experiment.

5.5.2 Data collection

The learned model is trained on *dedicated* training flights, flown under the uncompensated baseline of Section 4.2 and never part of the comparison: a grid of vertical separations, lateral offsets and speeds in the scenarios of the library, and randomised waypoint flights (Section 4.4). Keeping them separate means that the learned model is never evaluated on its own training data. The training labels are computed after each flight from the recorded signals (Section 3.3).

Separations are approached from large to small, and a tighter configuration is attempted only after the looser ones are clean.

The set must span both vertical and lateral excitation — A1/A2/A3 *and* A4/A5/A7 — for the reason given in Section 5.3. A collection campaign that omits the laterally excited scenarios cannot support the predictive strategies, and the omission is not recoverable without re-flying.

5.5.3 Comparison campaign

The study has two phases. The *first phase* is the two-vehicle campaign described in the rest of this section. The *second phase* flies the three-vehicle scenarios of Table 5.1, including the null scenarios C1–C3, and evaluates the transfer of the learned model from two to three vehicles (RQ3), with the same controllers, after the first phase; its cells and counts are fixed when the first phase is complete.

In both phases each strategy flies the same scenario settings. A *cell* is one scenario setting under one strategy, and its unit of analysis is one flight. The first tier of the first phase comprises twelve two-vehicle settings at a lower-vehicle height of 0.5 m (Table 5.2); with the four implemented strategies this gives 48 cells, flown until five clean flights per cell are obtained, 240 clean flights in all, with at most eight attempts per cell, a cell that does not reach five clean flights being reported as not reached. Further two-vehicle tiers are flown only if time remains before the second phase. Because the strategies are selected at run time (Section 4.1.5), they are flown in blocks within a session, the order of the blocks rotating between sessions, with flights of the baseline at the start and the end of every session against drift in the battery state, temperature and calibration.

Strategies 0–3 are compared under identical trajectories, formations, hardware and separations, and with one shared set of gains, filters and physical parameters (Section 4.7).

For the learned model two kinds of flight are distinguished. On *prediction-off* flights, those of Strategies 0 and 1, the prediction is recorded but not used, so predicted and measured residuals can be compared without the prediction having moved the vehicle. On *prediction-on* flights, those of Strategies 2 and 3, the prediction drives the controller.

5.5.4 Metrics

Three axes, corresponding to the research questions.

Notation and windows. For vehicle i, $p_{\mathrm{cmd},i}(t)$ is the commanded position recorded onboard and $p_i(t)$ the measured position, and $e_{p,i}(t) = p_{\mathrm{cmd},i}(t) - p_i(t)$ is its position error, with the sign of e_p in Chapter 4; e_1, e_2, e_3 are the unit vectors of the world frame. Each scenario defines an analysis window W, the set of samples in which its commanded geometry is in its nominal configuration, excluding take-off, landing and the settling intervals of the scenario definition. For the swap scenarios (A8, B3) an event window $W_c = [t_c - \Delta, t_c + \Delta]$ is centred on each crossing time t_c, defined as the instant of minimum *commanded* horizontal distance between the two vehicles concerned; because it is taken from the commanded trajectory, it does not depend on how a controller responded.

Confirm before the campaign: the source of p_i (onboard estimate or motion-capture pose, both recorded), the window boundaries per scenario, and the half-width Δ of the event window.

For a quantity q sampled at instants $t_k \in W$, $\mathrm{RMS}_W(q) = \left(\frac{1}{|W|}\sum_k \|q(t_k)\|^2\right)^{1/2}$.

Tracking error. Position RMSE, total and per axis, and the maximum deviation from the commanded trajectory: $\mathrm{RMS}_W(e_{p,i})$, $\mathrm{RMS}_W(e_{p,i}^\top e_\ell)$ for $\ell = 1, 2, 3$, and $\max_{t\in W}\|e_{p,i}(t)\|$, with the vertical maximum $\max_{t\in W}|e_{p,i}(t)^\top e_3|$ reported separately because downwash acts mainly vertically. Alongside these, the attitude tracking error, reported as the RMSE and peak of the geometric attitude error e_R. A compensator that improves position by working the attitude loop harder has not improved tracking so much as moved the cost, and only the attitude figure makes that visible. The controller records its actual and its desired attitude (Section 5.4.2); e_R is computed from them with the definition of Chapter 4, the same for every strategy, and $\mathrm{RMS}_W(e_R)$ and $\max_{t\in W}\|e_R(t)\|$ follow from it.

Residual rejection. The error in the *separation between vehicles*, measured against the commanded separation. This is a separation error, not the residual itself; the residual that remains is reported next to it. Downwash acts as a bias, the lower vehicle sags, widening the vertical gap, so the mean error is reported as the primary quantity and the spread as secondary. A compensator that works moves the mean back toward the commanded value. Reporting only the spread would hide precisely the effect being studied. For an upper vehicle u and a lower vehicle b in a stack, $e_{\mathrm{sep}}(t) = (p_u(t) - p_b(t))^\top e_3 - (p_{\mathrm{cmd},u}(t) - p_{\mathrm{cmd},b}(t))^\top e_3$; the reported quantities are its mean and standard deviation over W (or W_c).

Alongside it, the magnitude of the measured residual that remains after compensation, and for every strategy carrying a learned model, the prediction against the measurement. The remaining residual is $\mathrm{RMS}_W(a_{\mathrm{res}})$, reported with its vertical component. The prediction error of a model is evaluated on prediction-off flights as $\epsilon_{\mathrm{pred}} = \mathrm{RMS}_W(\hat{a}_{\mathrm{res},z} - a_{\mathrm{res},z})$ together with its mean, which separates a bias from scatter. The transfer penalty that RQ3 asks for is the difference between ϵ_{pred} on the three-vehicle scenarios and on the two-vehicle scenarios, for the same weights, absolute and relative to the two-vehicle value.

Fix which two-vehicle scenarios are paired with each three-vehicle scenario for the transfer penalty (for example B1 with A2 at matching height differences).

Robustness and cost. Control effort; behaviour as separation tightens; abort and failure rate; onboard computation per control step; sensing prerequisites; and the quantity

of training data required to reach a given prediction accuracy. Control effort is reported as RMS_W of the commanded torque and of the change of the motor commands between samples, and as the fraction of samples in which any motor command is at its limit.

Confirm the effort measures before the campaign.

5.5.5 Statistical treatment

Each metric is computed per flight, then summarised per (scenario, strategy) cell over the repetitions. Because strategies are interleaved within sessions, the comparison of a strategy with Strategy 0 can be made on pairs of flights from the same session. A difference between strategies is judged against the measurement floor in two steps (Section 5.3.1): for the first phase, provisionally against the repeat-to-repeat variation within a cell, between flights of the same strategy, scenario and setting; and finally, once the second phase has flown the null scenarios, against their floor.

Fix before the campaign: the summary statistic per cell, the paired test or interval method, the effect-size measure, and the rule by which a difference is compared with the provisional floor and with the measurement floor of the null scenarios.

5.6 The role of simulation

A software-in-the-loop simulator with the published Neural-Swarm2 interaction model [13] exists for the flight software. Its role in this thesis is deliberately limited.

Simulation is never used to validate a method. It cannot support a claim about how well a method works, for a reason specific to this subject: the interaction force in simulation is produced by a learned model of the same family as the one being fitted — in fact of the same architecture — and learning a network's output with another network is a substantially easier problem than learning real air.

No result in the two-robot results chapter or the multi-robot results chapter rests on simulation.

5.7 Threats to validity

Splitting the training and test data by contiguous flight segment rather than by sample deserves a note, because the obvious alternative is badly wrong here. At 500 Hz consecutive samples are nearly identical, so a random per-sample split places a near-duplicate of almost every test point in the training set and reports a validation error that means nothing. Neural-Swarm2 splits its data the same way, assigning pieces of about 20 s to training or validation rather than individual samples [13].

Two threats are not mitigated and are stated as limitations rather than managed. The results concern a single airframe; quantities are normalised where a normalisation is defensible, but generalisation to other platforms is argued rather than shown. And a slow drift of the thrust constant within a flight, for example from rotor wear or temperature, appears as a slow drift in the measured residual; it is not compensated, and the null scenarios and the baseline anchors bound it but do not remove it.

5.8 Summary

Identical brushless quadrotors, each carrying an optical rotor-speed sensor and an SD card, flying frozen scenarios inside a tracked volume of roughly 2 m × 4 m × 1.7 m, with the controller at 500 Hz and 96 variables recorded onboard at 500 Hz independently of team size. Strategies 0–3 are one controller with one shared gain set, frozen at the hardware gate and selected at run time in blocks within each session. The study has two phases: a two-vehicle campaign, whose first tier comprises 48 cells of five clean flights each, and then the three-vehicle scenarios, flown with the same controllers. The learned model is trained on dedicated flights that are never part of the comparison, and the null scenarios of the second phase establish what a measurement of zero looks like under the same pipeline; until then the two-vehicle results are read against the repeat-to-repeat variation within a cell.

Continue revising to past tense for each scenario and controller row as the comparison campaign's repetitions are completed. Record any deviation from the shared gain set, frozen after the hardware gate, or from scenario parameters in this chapter.

Table 5.1: The formation library. n is the default number of vehicles; separations are the defaults of the scenario definitions and can be varied as parameters. Scenarios marked † carry elevated risk and are gated behind an explicit acknowledgement; those marked $*$ are controls with no vertical overlap, in which no downwash is expected (C2 also probes the wake behind a vehicle). A6, C4 and C5 are defined in the library but are not part of the comparison campaign.

	Scenario	n	Default parameters	Purpose
Two vehicles				
A1	Vertical stack, hover	2	$\Delta z = 0.50\,m$, 12 s hold	Baseline. Nothing moves, so any relative displacement is interaction rather than tracking error
A2	Vertical stack, tracking	2	$\Delta z = 0.30\,m$, circle, 0.4 m/s	Constant separation while both vehicles move
A3	Static top	2	$\Delta z = 0.30\,m$, 0.4 m/s, 4 passes	Fixed wash source, lower vehicle sweeps through: captures the transition, not only the level
A4	Offset stack	2	$\Delta z = 0.60\,m$, offset 0.10 m, lemniscate	Partial overlap; asymmetric and hardest to model
A5	Reverse-circle stack	2	$\Delta z = 0.50\,m$, radius 0.40 m, 7.5 s period	Lateral offset swept twice per lap at constant height and speed
A6	Extreme stack †	2	$\Delta z = 0.10\,m$, 0.20 m/s	Strongest interaction, smallest error budget
A7	Dynamic merge †	2	Δz 1.10 m $\rightarrow$ 0.10 m, 0.30 m/s	One continuous sweep of the whole separation range: gives a model the gradient, not only levels
A8	Vertical swap	2	$\Delta z = 0.25\,m$, span 1.0 m, 6 s	Wash arrives as a transient with a known arrival time; hardest case for a feedforward model
Three vehicles				
B1	I-stack	3	Δz 0.20 m and 0.30 m, circle	Lowest vehicle in the combined wash of two
B2	V-stack	3	Δz 0.20 m and 0.30 m, offset 0.10 m	Asymmetric aggregation, which B1 cannot produce
B3	Vertical swap	3	$\Delta z = 0.22\,m$, span 0.55 m	Three crossings at three height differences in one flight
Controls and coverage				
C1	Side-by-side coplanar $*$	3	spacing 0.30 m, hover	A residual here, and not in A1, is not downwash
C2	Leader–follower line $*$	3	gap 0.50 m, 0.4 m/s	The wake behind a moving vehicle, not the downwash below it
C3	Equilateral triangle $*$	3	side 0.50 m, hover	Coplanar three-body control
C4	Docking approach †	2	lateral 1.0 m, Δz 0.50 m $\rightarrow$ 0.10 m	Simultaneous lateral and vertical closure
C5	Near-ground pass	1	height 0.15 m, 0.25 m/s	Ground effect alone, separable from downwash

Table 5.2: Settings of the first tier of the first, two-vehicle phase. Speeds are peak speeds of the lower vehicle.

Scenario	Setting	Settings
A1 vertical stack, hover	$\Delta z \in \{0.25, 0.5, 0.75\}$ m, 15 s hold	3
A8 vertical swap	$\Delta z = 0.5\,m$, speed $\in \{0.2, 0.3, 0.4, 0.5\}$ m/s	4
A2 stack tracking (circle)	$\Delta z = 0.5\,m$, 0.4 m/s	1
A3 static top	$\Delta z = 0.5\,m$, 0.4 m/s	1
A4 offset stack	$\Delta z = 0.5\,m$, offset 0.10 m, 0.4 m/s	1
A5 reverse-circle stack	$\Delta z = 0.5\,m$, 0.4 m/s	1
A7 dynamic merge	Δz 1.1 m $\rightarrow$ 0.1 m, 0.4 m/s	1
Total		12

Table 5.3: Threats to validity and the design decisions taken against them.

Threat	Mitigation	Residual risk
Tuning confound	One controller with one shared gain set for Strategies 0–3, frozen at the hardware gate; runtime selection allows blocks within a session	The shared gains may suit some switch combinations better than others (§4.7)
Measurement floor	Null scenarios C1–C3 bound it under the same pipeline (second phase); provisionally the repeat-to-repeat variation within a cell	First-phase results are read against a provisional floor until the second phase
Model error read as interaction	Hardware-gate flights at large separation (first phase); null scenarios (second phase); airframe constants identified once and frozen	A drift of the thrust constant within a flight is not compensated
Training/test leakage	Dedicated training flights, never part of the comparison; validation split by flight, never by sample	—
Session effects	Strategy blocks within a session, order rotated between sessions; baseline flights at start and end; repeated conditions	—
Silent loss of the residual	Rotor speeds and $a_{\text{res}} \neq 0$ checked at the start of every session	A rotor reading lost during a flight
Radio vs onboard clock	Dataset recorded onboard at 500 Hz; analysis uses the merged onboard logs and the commanded setpoint recorded onboard	Residual-time alignment across vehicles depends on merge quality; the merge refuses an alignment whose position fit to the vehicle's own commanded trajectory exceeds 15 cm RMS, unless overridden explicitly
Mocap identity / marker swap	Pre-takeoff check of the onboard estimate against motion capture; role verification from merge	Residual risk if a session bypasses the gate
Simulation-to-hardware gap	No claim rests on simulation	—
Single airframe	—	**Not mitigated.** Generalisation beyond this platform is discussed, not demonstrated

Chapter 6

Two-Robot Experimental Results

This chapter reports the first phase of the study, the systematic comparison on the two-vehicle team. Chapter 5 fixed the platform, scenarios, metrics and protocol; Chapter 4 defined Strategies 0–4, of which Strategies 0–3 are compared. The sections below follow the analysis from aggregate tracking metrics through control effort, the predictive residual path, and the attribution of effects to a loop, to a summary of when each strategy is preferable. Research questions RQ1, RQ2 and RQ4 (Section 1.5) are answered primarily here; RQ3 is deferred to the multi-robot results chapter. The null scenarios that set the measurement floor are flown in the second phase. In this chapter the results are therefore read against a provisional floor, the repeat-to-repeat variation within a cell (Section 5.5.5); the multi-robot results chapter reads them again against the floor of the null scenarios. Every "negative answer" below refers to the provisional floor.

All metrics are those defined in Section 5.5.4, computed per flight and summarised per (scenario, strategy) cell as specified in Section 5.5.5.

6.1 Experimental campaign overview

The comparison flights use two brushless vehicles, the study vehicle in the lower position of vertical stacks and a second vehicle on top, with external pose from a four-camera OptiTrack system and onboard logging at 500 Hz on micro-SD as the source of record (Section 5.1). Radio telemetry is for monitoring only; thesis metrics are computed from merged uSD logs after role alignment, using the same pipeline documented in Chapter 5.

Scenarios are drawn from the frozen formation library (Chapter 5, Section 5.3): the training and comparison matrix prioritises vertical separation and downwash-dominated geometries (e.g. A1, A3, A4, A7); the null coplanar controls (C1–C3) that bound the measurement floor are flown in the second phase. Each (scenario, strategy) cell is intended to be repeated n times under interleaved runtime controller selection so session effects do not align with strategy choice (Table 5.3).

The compared set follows Chapter 4: Strategy 0 (uncompensated geometric baseline), Strategy 1 (INDI), Strategy 2 (geometric + learned residual) and Strategy 3 (INDI + learned residual), one controller with one shared gain set frozen after the hardware gate. Indicative single flights and partial training-data collection exist; the tables and figures in the following sections are reserved for the completed comparison repeats.

PLACEHOLDER: Table — scenarios flown × strategies, with n per cell, flights excluded and the reason for each exclusion, and every deviation from the protocol of Section 5.5.

Table 6.1: Tracking error summary (two-robot comparison campaign).

Scenario	Strategy	RMSE x	RMSE y	RMSE z	Peak $\|\mathbf{e}\|$	n
— *populated from the per-flight metrics* —						
Placeholder	Strategy 0	—	—	—	—	—

Table 6.2: Control effort and safety margins (two-robot comparison campaign).

Scenario	Strategy	Effort proxy	Sat. fraction	Min sep. (m)	n
Placeholder	Strategy 0	—	—	—	—

6.2 Tracking and attitude

This section answers the tracking half of RQ1: whether compensating strategies reduce position and separation error relative to Strategy 0 under identical trajectories, and how reactive versus predictive families rank. Attitude excursions matter for safety and for interpreting residual magnitude; they are reported alongside position error rather than as a separate optimisation target.

PLACEHOLDER: Table 6.1. Columns: scenario, strategy, position RMSE per axis and total over the window W, maximum deviation, mean and standard deviation of the separation error e_{sep}, RMSE and peak of $\|e_R\|$, remaining $\text{RMS}_W(a_{\text{res}})$, n. Rows: all (scenario, strategy) cells flown. Statistic: the per-cell summary and the paired within-session difference to Strategy 0 fixed in Section 5.5.5, judged against the provisional floor by the rule fixed there, and again against the null-scenario floor in the multi-robot results chapter.

PLACEHOLDER: Figure (grouped bars) — strategies × metrics, one panel per metric family (position RMSE, separation error, attitude error), error bars as fixed in Section 5.5.5.

PLACEHOLDER: Figure (time series) — commanded and achieved separation for one representative transient scenario (A8 or A4), Strategies 0–3 on the same axes, crossing time from the commanded geometry marked.

A short subsection will contrast Strategy 0 with Strategy 1 (Section 4.3), the same controller with and without the measured residual.

Negative answer. If no compensating strategy differs from Strategy 0 beyond the provisional floor on the separation error (by the rule of Section 5.5.5), the answer to the tracking half of RQ1 is that, at the separations flown, compensation does not measurably improve tracking, and the section reports that rather than a ranking.

6.3 Control effort and safety

Better tracking is not automatically preferable if it requires sustained saturation or leaves less margin on separation. This section supports RQ4 by pairing performance with effort proxies and closest-approach statistics.

PLACEHOLDER: Table 6.2 — RMS commanded torque, RMS change of the motor commands, fraction of samples with a motor command at its limit, minimum achieved horizontal and vertical separation; per-cell summary and n as in Section 5.5.5.

The prose here will state explicitly when a strategy trades higher effort for lower tracking error, and when reduced separation margin disqualifies a configuration despite good RMSE.

6.4 Predictive strategy (Strategy 2)

The learned model is evaluated in two layers. On *prediction-off* flights, those of Strategies 0 and 1, the prediction is recorded but does not drive the loop; these flights measure model quality as the prediction error ϵ_{pred} of Section 5.5.4, independent of loop gain. On *prediction-on* flights, those of Strategies 2 and 3, the prediction drives the controller; Strategy 2 is compared with Strategy 0 on matched scenarios. RQ1 and RQ2 both depend on the second; the first explains it.

PLACEHOLDER: Figure — predicted against measured $a_{\text{res},z}$ on prediction-off flights (scatter, with bias and RMSE); second panel: error against relative geometry (lateral distance, Δz, whether the neighbour is inside the input gate). Training scenarios held out from each panel stated in the caption.

PLACEHOLDER: Table — Strategy 2 (prediction-on) against Strategy 0 on matched scenarios, same columns as Table 6.1; caption states the training scenarios and the weight set used.

Negative answer. A model whose prediction error on prediction-off flights is not smaller than the measured residual itself cannot be expected to help, and a prediction-on improvement within the provisional floor (Section 5.5.5) is reported as no improvement.

6.5 Loop attribution

Two checks connect the measured differences to the mechanisms of Chapters 3 and 4. First, the direction of the effect: Chapter 3 predicts that, with the residual subtracted, the formation-induced displacement in a quasi-static stack is smaller under Strategy 1 than under Strategy 0 during the transient, while the vertical integral that both share removes a constant offset in either. Second, the loop that carries an effect: the configurations with switch A or switch B alone (Section 4.1.5) are diagnostic, not strategies, and are used only to attribute a Strategy 1 effect to the force or the moment term.

PLACEHOLDER: Table — quasi-static stack (A1), separation error of Strategy 1 against Strategy 0, with the expected direction stated before the data; diagnostic configurations reported separately and labelled as diagnostic.

6.6 Hybrid strategy (Strategy 3)

Strategy 3 is compared with both of its parents on matched scenarios: with Strategy 1, which has the same measured terms, and with Strategy 2, which has the same prediction. Chapter 4 derives what to look for (Section 4.5): when the thrust has settled, the prediction acts on the measured-residual loop as an offset of its acceleration reference, which the vertical integral removes; where the prediction leads the filtered measurement in time, in the transient scenarios, Strategy 3 can differ from Strategy 1.

PLACEHOLDER: Table — Strategy 3 against Strategy 1 and against Strategy 2, same columns as Table 6.1, quasi-static settings (A1) and transient settings (A8) reported separately; figure: separation error against time around a crossing for Strategies 1–3.

Negative answer. If Strategy 3 does not differ from Strategy 1 beyond the provisional floor (Section 5.5.5), the prediction adds nothing to the measured terms at the settings flown, and the result is reported as such.

Table 6.3: Strategy suitability by scenario class (to be filled after the comparison campaign).

Strategy	Vertical stack	Lateral traverse	Near-ground	Fast transient
Strategy 0 (baseline)	—	—	—	—
Strategy 1 (INDI)	—	—	—	—
Strategy 2 (pred. + geo)	—	—	—	—
Strategy 3 (INDI + pred.)	—	—	—	—
Strategy 4 (FBL; not impl.)	—	—	—	—

6.7 Summary — when which strategy

This section closes the two-robot story for RQ1 and RQ2: a scenario-class breakdown (vertical stack, lateral crossing, near-ground, fast transient) with both qualitative ranking and the quantitative backing from Tables 6.1–6.2.

PLACEHOLDER: Summary table — rows: strategies; columns: scenario classes; cells: best tracking / effort trade-off / data and compute cost (RQ4 pointers). No ranking is entered without the statistical support of Section 5.5.5.

Train/test splits by scenario class (Section 5.7) feed RQ2 directly: a crossover in favour of reactive or predictive compensation should appear here if it exists in the data.

Negative answer. If no strategy is preferable in one class and not in another beyond the provisional floor (Section 5.5.5), RQ2 is answered as "no regime dependence detected at the separations and speeds flown", with the excitation actually achieved stated next to it.

Chapter 7

Multi-Robot and Scaling Results

This chapter reports the second phase of the study: the three-vehicle scenarios, flown with the same controllers after the two-vehicle campaign of the two-robot results chapter. Its cells and counts are fixed when the first phase is complete (Chapter 5). The phase has two purposes. It flies the null scenarios C1–C3, which set the measurement floor against which the two-vehicle results are finally read, and it answers RQ3 (Section 1.5), the transfer penalty when a summed per-neighbour residual model trained on two-vehicle data is applied to three-vehicle formations. The chapter also states how an extension to four or five vehicles would be structured, without treating that extension as part of the thesis.

7.1 Three-robot comparison campaign

Three-robot flights use three vehicles of the platform of Section 5.1 and scenarios B1, B2 and/or B3 from the frozen library (I-stack, V-stack, and merge geometries). The metric pipeline is intentionally identical to the two-robot results chapter: per-flight metrics, separation error, effort proxies, and residual logging under Strategy 0 versus compensating strategies where implemented for $N = 3$.

PLACEHOLDER: Table — B1/B2/B3 (subset as flown) × Strategies 0–3; same column headers as Table 6.1; n repeats per cell.

7.2 Null scenarios and the measurement floor

The null scenarios C1–C3 are flown in this phase. They give the measurement floor of Section 5.3.1, and the two-vehicle results, read provisionally against the repeat-to-repeat variation within a cell, are read here against it a second time.

PLACEHOLDER: Table — C1–C3 × strategies flown, residual and tracking metrics of Section 5.5.4, n; second table: the comparisons of the two-robot results chapter against this floor next to their provisional reading, and every case in which the two readings differ.

7.3 Transfer penalty for the summed predictor

RQ3 is not a test of whether aerodynamic interactions superpose; the literature already reports that they do not. The engineering question is how large the penalty is when the same onboard predictor, trained on two-robot data, is evaluated on three-robot relative configurations, and whether that penalty is small enough to keep the summed architecture. The

penalty is the difference in prediction error ϵ_{pred} between three-vehicle scenarios and their paired two-vehicle scenarios for the same weights, absolute and relative (Section 5.5.4), evaluated on prediction-off flights.

PLACEHOLDER: Figure/table — ϵ_{pred} on three-robot prediction-off flights against the paired two-robot scenarios; scenario-resolved; contribution of each neighbour's term next to the summed prediction where the logs allow it.

PLACEHOLDER: Closed-loop block — if Strategies 2 and 3 are flown on B-series, compare tracking and residual metrics to Strategy 0 on the same scenarios.

Negative answer. A penalty that the rule of Section 5.5.5 does not distinguish from the repetition-to-repetition variation of ϵ_{pred} on the paired two-vehicle scenario is reported as "no transfer penalty detected"; a detected penalty is reported with its size relative to the measured residual, which is what decides whether the summed model trained on two vehicles remains usable on three.

7.4 Extension to four and five vehicles

A sketch of scenarios with four or more vehicles describes how such formations would stress superposition and asymmetric wash without replacing the frozen scenario library. The sketch is a template only: its geometries must pass the pre-flight check and the supervisor's approval before any simulation or flight. No results from it are claimed here.

The primary thesis comparison remains two- and three-robot; n≥4 is documented so a future campaign can reuse the same logging, merge and analysis tools.

7.5 Limitation if three-robot data are incomplete

If the third vehicle arrives too late. RQ3 needs flights with three vehicles, because the effect it asks about, the overlap of two wakes, does not exist in two-vehicle data. If the third vehicle is not available in time, RQ3 is not answered on hardware, and the thesis says so. Two-vehicle logs can only check how the network behaves on one neighbour at a time. They say nothing about the summed prediction for two neighbours. Without the null scenarios, the two-vehicle results keep their provisional floor, the repeat-to-repeat variation within a cell, and are reported with that qualification.

The limitation is stated plainly: without repeated three-robot compensated flights, the thesis cannot rank the strategies at $N = 3$ with the same confidence as at $N = 2$; the summed-architecture decision rests on the magnitude of the open-loop penalty alone, and generalisation to larger teams remains future work (Section 9.4).

7.6 Chapter summary

PLACEHOLDER: One paragraph summarising the RQ3 answer once metrics exist — penalty size, whether the summed predictor remains defensible, link back to Section 6.7 for two-robot vs three-robot consistency.

Chapter 8

Discussion

The two-robot and multi-robot results chapters report measurements; this chapter interprets them against the research questions in Section 1.5, checks the intended contributions in Section 1.6, and revisits the threats to validity flagged in Section 5.7. A negative answer to any question is treated as a result, not as a failed experiment.

8.1 RQ1 — Reactive versus predictive compensation under identical conditions

RQ1 asks, under identical hardware, trajectories, formations and separations, how reactive and predictive interaction-force compensation compare to the uncompensated baseline in trajectory-tracking accuracy and residual-force rejection. Evidence comes from Section 6.2 and residual magnitudes on the same flights. If compensating strategies do not separate from one another but all beat Strategy 0, the practical conclusion shifts toward "compensation matters; family matters less," and RQ4 dominates the engineering decision. If they separate strongly, this chapter must explain whether the gap is robust across repeats or driven by a single scenario class.

PLACEHOLDER: Interpretation paragraph once the comparison tables exist — ranking of the implemented strategies, residual rejection, statistical spread across n.

8.2 RQ2 — Regime dependence

RQ2 asks whether the relative advantage of reactive versus predictive compensation depends on separation, relative velocity, and quasi-static versus transient interaction. The scenario library was designed to isolate these regimes (Section 6.7); a crossover supports conditional guidance ("use predictive in static stacks, reactive in fast crossings"). If no crossover appears, the discussion must say so and attribute flat rankings to measurement floor, tuning, or insufficient excitation rather than silently claiming regime independence.

PLACEHOLDER: Scenario-class breakdown referencing Table 6.3 and train/test splits across classes.

8.3 RQ3 — Two-robot training on three-robot formations

RQ3 asks how large the transfer penalty is for a summed per-neighbour model trained on two-vehicle data when used on three-vehicle formations, and whether that penalty is small

enough to keep the summed architecture. The question presupposes non-superposition; the contribution is quantitative. Section 7.3 supplies the numbers; here they are interpreted against deployment cost (single weight set, permutation invariance) versus retraining or per-formation models.

PLACEHOLDER: Interpret open-loop and (if available) closed-loop three-robot penalty; tie to Section 7.5 if only logs exist.

8.4 RQ4 — Cost versus performance

RQ4 asks, for each compared strategy, what sensing, onboard compute and training data are required, and how those costs sit next to tracking and rejection. Section 6.3 and the predictive-path subsection supply performance-side numbers; cost includes the rotor-speed source, onboard logging, how the network weights are deployed, the tuning time of the shared gain set (Table 5.3), and the training-data collection that Strategies 2 and 3 require.

PLACEHOLDER: Cost–performance synthesis table (qualitative + quantitative where measured); no invented rankings.

8.5 Contributions revisited

Section 1.6 listed three intended contributions: a controlled comparison, a quantification of the summed-model transfer penalty, and an engineering account of cost alongside performance. The apparatus (scenario library, residual path, training pipeline) was explicitly excluded as a scientific claim. For each item below, this section will state whether the comparison campaign achieved it, partially achieved it, or could not be tested, without upgrading apparatus description into a result.

PLACEHOLDER: Contribution (1) — controlled comparison achieved? cite the two-robot results chapter, its scope and null scenarios.

PLACEHOLDER: Contribution (2) — transfer penalty quantified? cite the multi-robot results chapter.

PLACEHOLDER: Contribution (3) — cost account complete? cite RQ4 subsection.

8.6 Threats to validity, revisited

Section 5.7 and Table 5.3 listed threats and mitigations at design time. After flights, the discussion closes the loop: which mitigations held in practice, which residual risks materialised, and which limitations remain unavoidable.

Tuning and fairness. One shared gain set, frozen before the comparison, is necessary for RQ1; the discussion will report whether any comparison flight required ad-hoc gain changes and, if so, exclude or flag those cells, and whether the shared gains appear to have disadvantaged one switch combination.

Measurement floor and null scenarios. C1–C3 bound how small a reported improvement can be before it is indistinguishable from noise; revisiting this prevents over-interpreting marginal RMSE gaps. Because they are flown in the second phase, the

discussion states for each two-vehicle result whether its provisional reading against the repeat-to-repeat variation and its final reading against the null-scenario floor agree.

Single airframe and session effects. These were not mitigated by design; results are scoped to the Crazyflie brushless configuration flown. Interleaving reduces but does not remove session effects; repeat variance in the two-robot results chapter quantifies what remains.

Simulation. No thesis claim rests on simulation (Section 5.7); any simulation-only diagnosis (for example the choice of interaction model in the simulator) is mentioned only as implementation context, not as validation of a strategy.

PLACEHOLDER: Short paragraph per threat row in Table 5.3 — "after data: mitigated / observed / not testable" once the comparison is complete.

8.7 The modelling assumptions, revisited

Chapter 3 listed the assumptions the residual formulation rests on and where each is expected to fail (Table 3.1), so that a negative result can be read as evidence about an assumption rather than as an unexplained failure. This section returns to each row: whether the data are consistent with it, contradict it, or cannot test it.

PLACEHOLDER: One paragraph per row of Table 3.1, each naming the result that bears on it (null scenarios for model error; three-robot results for summation; prediction-off flights for the input set and the gate).

8.8 The uncompensated moment residual

Every strategy compensates the residual force only and leaves the residual moment to the attitude loop (Section 3.2). The cost of that choice is visible, if anywhere, in the attitude error and the control effort of the compensating strategies relative to Strategy 0.

PLACEHOLDER: Interpretation of $\|e_R\|$ and torque effort across strategies on the scenarios with lateral offset (A4, A5), where the interaction moment is expected to be largest.

8.9 Frozen gains and the scope of the comparison

Strategies 0–3 share one controller and one frozen gain set (Section 4.7). A difference between them is therefore a difference between switch combinations of that controller, not between the families in the abstract, and it is read against a gain set that was not chosen for any one combination. This section states what that implies for the generality of the ranking, what the interaction of the two estimates in Strategy 3 implies (Section 4.5), and what the absence of Strategy 4 implies for the predictive family (Section 4.6).

PLACEHOLDER: Interpretation once the comparison exists; any cell flown with changed gains is reported here.

Chapter 9

Conclusion

9.1 Summary

This thesis set out to compare interaction-force compensation strategies for tight multi-rotor formation flight on a common platform, trajectory set and logging pipeline, holding reactive and predictive families under the same residual definition and null-flight measurement floor. Strategies 0–3, the baseline, the reactive, the predictive and the hybrid strategy, are one controller with three switches, and Strategy 4 is specified but not implemented, in Chapters 3–4; the experimental campaign is described in Chapter 5 and reported in two phases, the two-vehicle phase in the two-robot results chapter and the three-vehicle phase in the multi-robot results chapter. The comparison is deliberately conditional: the useful outcome is guidance on when to pay for prediction, measurement, or neither, not a claim of universal superiority.

9.2 Answers to the research questions

RQ1.

PLACEHOLDER (RQ1): One-sentence headline answer — for each compensating strategy, whether it differs from Strategy 0 beyond the measurement floor (provisional, then that of the null scenarios) in separation error and tracking error, and in which direction; or that no difference was detected (Section 6.2).

RQ2.

PLACEHOLDER (RQ2): One-sentence headline — the scenario classes in which the reactive and the predictive strategy order differently, or that no regime dependence was detected at the separations and speeds flown (Section 6.7).

RQ3.

PLACEHOLDER (RQ3): One-sentence headline — the transfer penalty in prediction error, absolute and relative, and whether the summed architecture remains justified; or that RQ3 was not answered on hardware (Sections 7.3 and 7.5).

RQ4.

PLACEHOLDER (RQ4): One-sentence headline — cost against performance across the implemented strategies (sensing, compute, data, tuning) (Section 8.4).

9.3 Engineering takeaway

The engineering reader should leave with a decision rule, not only a ranking. RQ4 frames that rule: given brushless Crazyflies with RPM sensing and offline-trained predictors, which strategy family delivers acceptable tracking and residual rejection per unit of tuning effort, flight time for data collection, and onboard resource? The filled discussion (Section 8.4) will state that rule explicitly; until the comparison is complete, no rule is asserted here.

9.4 Future work

Several extensions are scoped and do not depend on the comparison outcome:

- **Larger teams.** The three-vehicle scenarios are the second phase of this thesis (Section 7.5 covers the case that it is incomplete); sketches for four or more vehicles remain a sketch until the core library and the comparison are finished.
- **A retrained single-vehicle hybrid.** The learned variant of the incremental hybrid of Cobo-Briesewitz et al. is not part of the compared set and remains a candidate extension.
- **Residual reinforcement-learning strategies.** Reinforcement-learning residual policies are not part of this comparison and would require a separate safety and data budget before any flight claim.
- **Onboard prediction beyond this platform.** The comparison evaluates the learned term on one airframe; other airframes and larger teams are open.

9.5 Closing remark

Whether compensation families separate sharply or collapse to "any compensation helps," the thesis is designed so either outcome is informative. The remaining work is to replace the placeholders in results and discussion chapters with measured outcomes from the lab campaign.

Bibliography

[1] Leonard Bauersfeld, Koen Muller, Dominic Ziegler, Filippo Coletti, and Davide Scaramuzza. Robotics meets fluid dynamics: A characterization of the induced airflow below a quadrotor as a turbulent jet. *IEEE Robotics and Automation Letters*, 10(2):1241–1248, 2025. arXiv:2403.13321v3.

[2] Kong Yao Chee, Pei-An Hsieh, George J. Pappas, and M. Ani Hsieh. Flying quadrotors in tight formations using learning-based model predictive control. arXiv:2410.09727, 2024.

[3] Eckart Cobo-Briesewitz, Khaled Wahba, and Wolfgang Hönig. Learned incremental nonlinear dynamic inversion for quadrotors with and without slung payloads. In *Proceedings of the 8th Annual Conference on Learning for Dynamics and Control (L4DC)*, Proceedings of Machine Learning Research, 2026. arXiv:2503.09441v2.

[4] Jennifer Gielis, Ajay Shankar, Ryan Kortvelesy, and Amanda Prorok. Modeling aggregate downwash forces for dense multirotor flight. arXiv:2312.03488, 2023.

[5] Weibin Gu, Jiance Zhao, and Alessandro Rizzo. Learning uncertainties online for quadrotor flight control: A comparative study. *Journal of Intelligent & Robotic Systems*, 111:98, 2025.

[6] Pei-An Hsieh, Kong Yao Chee, and M. Ani Hsieh. Online adaptation for flying quadrotors in tight formations. arXiv:2506.17488, 2025.

[7] Pei-An Hsieh, Fengjun Yang, Nikolai Matni, and M. Ani Hsieh. Flatness-preserving residual learning for real-time tight quadrotor formation flight. arXiv:2607.12275, 2026.

[8] Anoop Kiran, Nora Ayanian, and Kenneth Breuer. Influence of static and dynamic downwash interactions on multi-quadrotor systems. arXiv:2507.09463, 2025.

[9] Taeyoung Lee, Melvin Leok, and N. Harris McClamroch. Control of complex maneuvers for a quadrotor uav using geometric methods on se(3). arXiv:1003.2005, 2010. Companion of the CDC 2010 paper; version 4 (2011) held and used for all equation checks (attitude error function eq. 6, errors eqs. 8–9, torque law eqs. 11/20, Propositions 3–4).

[10] Taeyoung Lee, Melvin Leok, and N. Harris McClamroch. Geometric tracking control of a quadrotor UAV on SE(3). In *49th IEEE Conference on Decision and Control (CDC)*, pages 5420–5425, 2010. PRIMARY reference for the geometric SE(3) controller (Strategy 0). Stability regions (exponential below 90 degrees, almost-global attractiveness below 180 degrees) checked against the v4 companion (Propositions 3–4 there); numbering in this paper not checked.

[11] Jinjie Li, Liang Han, Haoyang Yu, Yuheng Lin, Qingdong Li, and Zhang Ren. Nonlinear MPC for quadrotors in close-proximity flight with neural network downwash prediction. In *62nd IEEE Conference on Decision and Control (CDC)*, pages 2122–2128, 2023. arXiv:2304.07794.

[12] Ajay Shankar, Heedo Woo, and Amanda Prorok. Docking multirotors in close proximity using learnt downwash models. In *International Symposium on Experimental Robotics (ISER)*, pages 427–437. Springer, 2023. arXiv:2311.13988.

[13] Guanya Shi, Wolfgang Hönig, Xichen Shi, Yisong Yue, and Soon-Jo Chung. Neural-swarm2: Planning and control of heterogeneous multirotor swarms using learned interactions. *IEEE Transactions on Robotics*, 38(2):1063–1079, 2022. arXiv:2012.05457.

[14] Guanya Shi, Wolfgang Hönig, Yisong Yue, and Soon-Jo Chung. Neural-swarm: Decentralized close-proximity multirotor control using learned interactions. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 3241–3247, 2020. arXiv:2003.02992.

[15] Ewoud J. J. Smeur, Guido C. H. E. de Croon, and Qiping Chu. Cascaded incremental nonlinear dynamic inversion control for mav disturbance rejection. arXiv:1701.07254, 2017.

[16] H. Smith, A. Shankar, J. Gielis, J. Blumenkamp, and A. Prorok. So(2)-equivariant downwash models for close proximity flight. arXiv:2305.18983, 2023.

[17] Ezra Tal and Sertac Karaman. Accurate tracking of aggressive quadrotor trajectories using incremental nonlinear dynamic inversion and differential flatness. *IEEE Transactions on Control Systems Technology*, 29(3):1203–1218, 2021. Extended version of the CDC 2018 paper; arXiv:1809.04048v2.

[18] Fengjun Yang, Jake Welde, and Nikolai Matni. Learning flatness-preserving residuals for pure-feedback systems. arXiv:2504.04324v3, 2025. A conference version appeared at IEEE CDC 2025, pp. 3035–3042; the extended arXiv v3 is cited.

[19] Ruiqi Zhang, Dingqi Zhang, and Mark W. Mueller. Proxfly: Robust control for close proximity quadcopter flight via residual reinforcement learning. arXiv:2409.13193, 2024.